Ostbayerische Technische Hochschule Amberg-Weiden
Fakultät Elektrotechnik, Medien und Informatik

Studiengang Bachelor Künstliche Intelligenz

Bachelorarbeit

von

Sebastian **Weidner**

Imitation Learning using Video-Conditioned Policies for Visually Guided Robotic Behavior

Imitation Learning unter Verwendung videokonditionierter Policies für visuell gesteuertes Roboterverhalten

Ostbayerische Technische Hochschule Amberg-Weiden
Fakultät Elektrotechnik, Medien und Informatik

Studiengang Bachelor Künstliche Intelligenz

Bachelorarbeit

von

Sebastian **Weidner**

Imitation Learning using Video-Conditioned Policies for Visually Guided Robotic Behavior

Imitation Learning unter Verwendung videokonditionierter Policies für visuell gesteuertes Roboterverhalten

Bearbeitungszeitraum: von 20. Januar 2026
bis 19. Juni 2026

1. Prüfer: Prof. Dr.-Ing. Christian Bergler

2. Prüfer: Prof. Dr. phil. Tatyana Ivanovska

OTH Amberg-Weiden

Elektrotechnik, Medien und Informatik

Eigenständigkeitserklärung gemäß § 27 (8) ASPO

Name und Vorname der Studentin/des Studenten: **Weidner, Sebastian**

Studiengang: **Bachelor Künstliche Intelligenz**

Ich bestätige, dass ich die Bachelorarbeit mit dem Titel:

Imitation Learning unter Verwendung videokonditionierter Policies für visuell gesteuertes Roboterverhalten

selbständig verfasst, noch nicht anderweitig für Prüfungszwecke vorgelegt, keine anderen als die angegebenen Quellen oder Hilfsmittel benutzt sowie wörtliche und sinngemäße Zitate als solche gekennzeichnet habe.

Datum: 19. Juni 2026

Unterschrift:

OTH Amberg-Weiden

Elektrotechnik, Medien und Informatik

Bachelorarbeit Zusammenfassung

Student:	**Weidner, Sebastian**
Studiengang:	Bachelor Künstliche Intelligenz
Professor:	Prof. Dr.-Ing. Christian Bergler
Durchgeführt in Hochschule:	OTH Amberg-Weiden
Betreuer in Hochschule:	Prof. Dr.-Ing. Christian Bergler
Ausgabedatum:	20. Januar 2026
Abgabedatum:	19. Juni 2026

Title:

Imitation Learning using Video-Conditioned Policies for Visually Guided Robotic Behavior

Abstract

The overarching goal of robotics is to develop a general-purpose robot capable of performing a wide range of tasks in unfamiliar real-world environments. The recent success of large language models has demonstrated that training on large and diverse datasets can lead to substantial performance improvements. These models already exhibit remarkable capabilities and are able to solve tasks for which they were not explicitly trained. In light of these advances, the scientific community is increasingly striving to replicate these successes in robotics. However, achieving this goal requires large and diverse robot-specific datasets, which are currently available only to a limited extent. This thesis provides an introduction to the field of intelligent robotics and examines various approaches to how robots can learn from human demonstration videos through imitation learning. The objective is to leverage the vast amount of video data available on the Internet. In particular, this work addresses how relevant knowledge can be extracted from Internet videos and how robots can benefit from such knowledge. Furthermore, the thesis analyzes existing challenges and discusses future research directions for the presented approaches.

Keywords

Robotics, Imitation Learning, Reinforcement Learning, VLA Models

Titel:

Imitation Learning unter Verwendung videokonditionierter Policies für visuell gesteuertes Roboterverhalten

Zusammenfassung

Das übergeordnete Ziel der Robotik ist die Entwicklung eines Allzweckroboters, der eine Vielzahl von Aufgaben in unbekannten realen Umgebungen ausführen kann. Durch den aktuellen Erfolg großer Sprachmodelle wurde gezeigt, dass das Training mit großen und vielfältigen Datenmengen zu enormen Performancefortschritten führen kann. Diese Modelle besitzen bereits heute bemerkenswerte Fähigkeiten und können Aufgaben bewältigen, für die sie ursprünglich nicht trainiert worden sind. In Anbetracht dieser Fortschritte, strebt die Wissenschaft zunehmend danach, diese Erfolge in der Robotik zu wiederholen, wofür aber große und vielfältige Mengen an spezifischen Roboter-Datensätzen erforderlich sind. Diese sind aber nur im begrenzten Umfang verfügbar. Diese Arbeit gibt eine Einführung in das Thema intelligente Robotik und untersucht verschiedene Ansätze, wie Roboter mithilfe von Imitation Learning von menschlichen Demonstrationsvideos lernen können. Ziel ist es, von den im Internet verfügbaren Videos zu profitieren. Insbesondere werden die Fragen beantwortet, wie relevantes Wissen von Internetvideos extrahiert werden kann und wie Roboter von diesen Wissen profitieren können. Zusätzlich werden bestehende Herausforderungen und zukünftige Forschungsrichtungen der vorgestellten Ansätze analysiert.

Schlüsselwörter

Robotik, Imitation Learning, Reinforcement Learning, VLA Models

Contents

Chapter 1

Introduction

1.1 Motivation

Robots are expected to play a key role in a wide range of application domains in the future. Already today, the use of robotic systems in industrial production is steadily increasing, as they can efficiently and reliably perform monotonous, simple, and repetitive tasks [1]. Despite the often substantial acquisition costs, an increasing number of companies are investing in the integration of robots into their production lines. The primary advantages of robotic systems lie in their high availability and predictability. Once deployed, only relatively low operating costs remain, which are mainly limited to energy consumption and maintenance expenses. Furthermore, robots can operate continuously, 24 hours a day and seven days a week, while maintaining consistent quality. This enables high process stability and significant productivity gains. Unlike human workers, robots do not become ill, participate in labor strikes, or require additional compensation for night shifts, weekends, or public holidays [2].

In general, robotic systems can be divided into two categories based on their programming paradigm [3]. In industrial practice, statically programmed robots currently dominate. These robots are designed for clearly defined tasks in structured environments. They follow predetermined motion sequences and can adapt only to a limited extent to their environment or task through the use of integrated intelligent sensors. Such systems are highly efficient in standardized production processes but reach their limits when environmental conditions or task requirements change. In contrast, intelligent robotic systems are becoming increasingly important. These systems are capable of responding to changes in their environment and adapting their behavior accordingly. In particular, the integration of artificial intelligence has opened up new possibilities for solving complex and unstructured tasks.

However, major challenges remain in the robust execution of diverse scenarios and in the ability to generalize to new objects, environments and tasks [4]. Today's intelligent robotic systems, and especially their underlying AI models, are often highly specialized and trained for a particular task and target environment [4]. Tasks that are not represented in the training data frequently cannot be performed reliably. Similar

difficulties arise when learned tasks must be executed in fundamentally different environments. The reason lies in the way the underlying AI models are trained: training datasets are typically tailored to a specific task and environment [5]. Increasing data diversity often introduces a trade-off, as it may lead to reduced accuracy in the execution of individual tasks.

Even if the goal is to develop a large, general-purpose robotic AI model, the lack of suitable and diverse robotic training data remains a significant obstacle [5]. Traditional task-specific approaches are reaching their limits due to their lack of scalability and flexibility. Consequently, current research aims to equip robots with visual and semantic understanding, enabling them to infer actions from high-dimensional inputs such as videos and images [5, 6]. This approach allows new sources of training data to be utilized. With visual and semantic understanding, for example, human demonstration videos from platforms such as YouTube can be incorporated into the training process. These videos contain a wide variety of actions and task executions that would be essential for training a generalizable robotic AI model [5]. In this context, imitation learning represents a promising approach and is therefore investigated in greater detail throughout this thesis.

1.2 Project Goal

The aim of this thesis is to provide an introduction to the field of robotics with a particular focus on imitation learning. The work is structured into two main parts.

The first part presents the broader context and motivation of general-purpose robotics, including the long-term goal of developing robots capable of operating in diverse and unstructured environments. It further discusses the key challenges associated with this objective, particularly the limitations of current data-driven approaches and the difficulty of acquiring large-scale robotic training datasets. Building on this foundation, the thesis analyzes approaches how to extract knowledge from demonstration videos and transferring it to robotic systems. Various methods and techniques are presented and analyzed, with a specific emphasis on imitation learning. Subsequently outstanding challenges are highlighted and future research directions with a view to developing an all-round robot. Subsequently, open challenges are outlined and future research directions are proposed toward the development of a general-purpose robotic system. Finally, to gather a better understanding of the presented theroie, selected research directions are discussed in greater depth.

The second part of the thesis focuses on the practical implementation of an open-source imitation learning model on a real robotic arm. This includes the initial setup and commissioning of the robot, as well as the integration of a machine learning model for execution on the robotic system. The objective is to enable the robot to learn and perform simple manipulation tasks, such as stacking cubes, solely based on human demonstration videos.

1.3 Outline of the Thesis

Chapter 2 introduced the fundamental concepts of imitation learning and distinguished between two major paradigms: Behavior Cloning and Learning from Videos (LfV).

Chapter **??** discussed the overarching objective of robotics research: the development of generalist robots capable of performing a wide range of tasks in unstructured and previously unseen real-world environments. A major obstacle to this vision is the robotics data bottleneck, which arises from the limited availability of large, diverse, and high-quality robotic datasets. A prerequisite for effective human-robot interaction is the ability to communicate goals and intentions to robotic agents. Therefore, chapter **??** examined how robots can be instructed and guided to perform desired tasks.

Chapter **??** focused on the central research questions of Learning from Videos: how relevant knowledge can be extracted from internet videos and how this knowledge can subsequently be transferred to robotic systems for policy learning and control.

Chapter **??** explored the role of robust video understanding in Imitation Learning (IL). Learning manipulation skills from video requires more than simple action recognition: (1) It demands meaningful representations of the environment, (2) an understanding of object affordances and human-object interactions, (3) recognition of human activities, and (4) accurate modeling of fine-grained hand movements. These capabilities form the basis for a general-purpose robot that can interpret human demonstrations.

Chapter **??** focused on approaches for learning coherent robot models from video data. (1) The chapter first introduced foundational perception and representation learning methods and techniques. (2) It then discussed both RL-based and imitation learning approaches (3) Hybrid methods, which combine the structured guidance and sample efficiency of imitation learning with the exploration and optimization capabilities of reinforcement learning. Special attention was given to the intersection of Learning from Videos (LfV) and Reinforcement Learning (RL), demonstrating how knowledge extracted from videos can be used to improve policy learning and enhance the sample efficiency of RL algorithms. (4) Last but not least, the chapter presented multi-modal learning approaches and Vision-Language-Action (VLA) models and the three Architecturial paradigms, including sensorimotor, world-model-based, and affordance-based architectures.

Chapter **??** examined the major challenges and future research directions in IL and video-based robotic learning.

Finally in Chapter **??** of the literature overview, selected IL approaches deployed on real robotic systems were examined to provide practical insight into the theoretical concepts discussed throughout the thesis.

The second part of the thesis focused on the practical implementation of an IL approach on a real robotic system, which is examined in Chapter **??**.

Chapter 2

Foundations

This chapter provides an overview of the key concepts and theories that form the foundation of the following chapters.

2.1 Glossary

Robot Trajectory:

A robot trajectory describes how the robot moves from a starting state to a target state, including position, orientation and, if applicable, speeds, accelerations, and forces acting on the robot. A robot trajectory is a temporal sequence of actions and the resulting robot states.

Inference:

Inference refers to the process of using a trained model to make predictions or decisions based on new, unseen data. In the context of robotics, inference involves the robot applying its learned policy in real world or simulation to execute a given task.

Annotation:

Annotating refers to the process of labeling data, in which data is enriched with additional information. For example, this may include assigning class labels, marking target objects in images, or labeling states and actions within robot trajectories. Annotated, or labeled, data forms the foundation for supervised learning approaches.

Weakly Supervised Learning:

Weakly supervised learning is a Machine Learning (ML) approach that utilizes limited or incomplete labeled data for training. In this context, the model learns from a combination of labeled and unlabeled data, or from data with noisy or imprecise labels. This approach is particularly relevant in robotics, where obtaining fully annotated data can be costly and time-consuming. Weakly supervised learning allows models to leverage large amounts of unlabeled data while still benefiting from the guidance provided by the available labeled data.

Self-Supervised Learning:

Self-Supervised Learning is a ML approach where the model learns to predict parts of the input data from other parts of the same data. This method does not require external labels, as the model generates its own supervisory signal from the data itself. For example, Large Language Models (LLMs) learn to predict the next word or masked words in a sentence based on the surrounding context, where the correct target words are already contained within the training data. Similarly, in computer vision, a model may learn to reconstruct masked regions of an image, with the original image providing the target information.

2.2 Machine Learning: Core Categories

There are various approaches to training an AI model for a robot. These involve methods from the four core ML categories, which can also be combined.

Supervised Learning:

In supervised learning, a model is trained using labeled training data. This means that, for the input data, the output data to be predicted are known. The goal is to train a model that correctly predicts the output data and makes reliable predictions on unseen data.

Unsupervised Learning:

In contrast to supervised learning, unsupervised learning uses unlabeled data for model training. The model tries to recognize structures, patterns or relationships in the data on its own.

Semi-Supervised Learning:

Semi-supervised learning is a combination of supervised and unsupervised learning. In this approach, a model is trained using a small amount of labeled data and a large amount of unlabeled data. The goal is to leverage the advantages of both approaches, particularly when annotating data is time-consuming or cost intensive. This is relevant in robotics, as the data used in that field is often difficult to label.

Reinforcement Learning:

RL differs fundamentally from the previously mentioned approaches. Instead of learning from fixed datasets, an agent actively interacts with its environment and tries to find out an optimal strategy to reach a defined goal. Through its actions, it influences the state of the environment and subsequently receives feedback in the form of rewards or penalties, see Figure 2.1 for a short overview. In the context of robotics, the robot is the agent. The objective is to learn a strategy (= policy) that maximizes the cumulative reward over time for the given task.

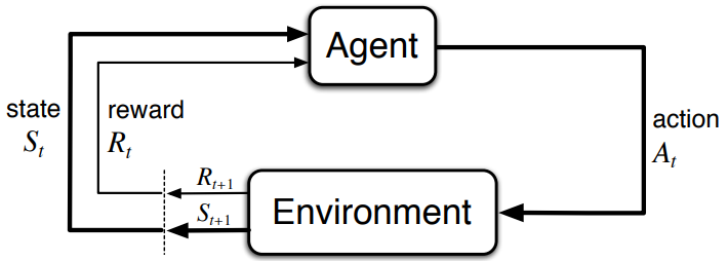


Figure 2.1: Base concept of Reinforcement Learning. Image source: [7].

The agent proceeds in its environment according to the trial-and-error principle to learn this optimal policy. Safety is a critical concern during training, as the agent

may execute unpredictable actions while exploring the state space. For this reason, simulation environments are a central component of training and are frequently used for pre-training the policy. The policy is then subsequently often fine-tuned in the real-world environment. Because the agent learns through trial and error, more efficient, unknown, and creative solutions can be found that a human might not have come up with. See section 2.8 for more details.

2.3 Neural Networks

Artificial Neural Networks are computational models inspired by the structure and functionality of biological brain. They are designed to learn complex relationships from data and have become a fundamental component of modern machine learning. Neural Networks are capable of approximating highly nonlinear functions and are therefore widely used in applications such as image recognition, natural language processing, forecasting, and control systems. There exist various types of neural network architectures, including Convolutional Neural Networks (CNNs), Recurrent Neural Networks (RNNs), or Transformer models, each tailored to specific types of data and tasks. The following sections provide an overview of the basic building blocks of neural networks, their mathematical formulation, and the training process.

2.3.1 Neuron

The basic element of an artificial neural network is the artificial neuron. The concept is motivated by biological neurons in the human brain, which communicate through electrochemical signals. In a simplified model, a biological neuron receives signals from other neurons through dendrites, processes the received information, and transmits an output signal through its axon [8]. Artificial neurons mimic this behavior mathematically by combining several input values into a single output.

History

The first mathematical model of an artificial neuron was proposed by McCulloch and Pitts in 1943 [9]. Their model consisted of a binary threshold unit that produced an output only when the weighted sum of its inputs exceeded a predefined threshold. Although this model did not include a learning mechanism, it established the foundation for later neural network research.

A major extension of this idea was introduced by Rosenblatt in 1958 with the perceptron [10]. In contrast to the McCulloch–Pitts neuron, the perceptron introduced trainable weights and a learning rule, making it possible to adapt the model parameters directly from data. However, the perceptron still relied on a simple threshold-based activation, which produces a linear decision boundary and was therefore limited to linearly separable classification problems [11].

This limitation was highlighted by Minsky and Papert, who showed that a single perceptron cannot represent functions such as XOR [11]. The development of multilayer

neural networks together with differentiable activation functions later overcame this restriction. These advances made it possible to train networks with multiple layers and enabled the modern neural network architectures that are used in deep learning today.

Mathematical Formulation

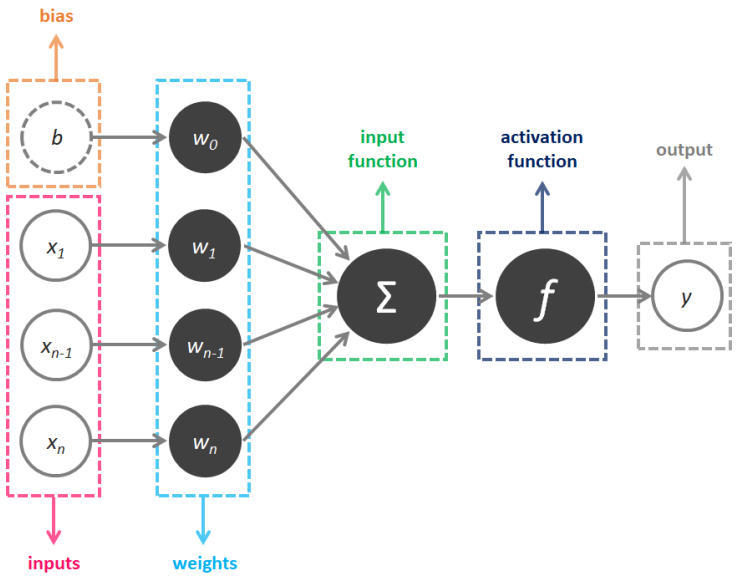


Figure 2.2: Perceptron of an Artificial Neural Network. Image source: [12].

A neuron receives an input vector $\mathbf{x} = [x_1, x_2, \ldots, x_n]^T$, where each vector element x_i is associated with a trainable weight w_i from the corresponding weight vector $\mathbf{w} = [w_1, w_2, \ldots, w_n]^T$. The neuron first computes a weighted sum of its inputs and adds a bias term b, see Figure 2.2. The resulting pre-activation value is given by:

$$z = \sum_{i=1}^{n} w_i x_i + b \tag{2.1}$$

The value z represents a linear combination of the input features. To enable the modeling of non-linear relationships, an activation function $\phi(\cdot)$ is subsequently applied, yielding the neuron's output:

$$y = \phi(z) = \phi\left(\sum_{i=1}^{n} w_i x_i + b\right) \tag{2.2}$$

Without the activation function, a composition of multiple neurons would still correspond to a single linear transformation and therefore could not represent complex non-linear patterns. Common activation functions used in modern neural networks include for example the GeLU, tanh (hyperbolic tangent), and ReLU.

The weighted sum in Equation 2.1 can be expressed more compactly using vector notation and ending in the form of:

$$y = \phi\Big(\mathbf{w}^T\mathbf{x} + b\Big) = \phi\left(\begin{bmatrix} w_1 & w_2 & \cdots & w_n \end{bmatrix}\begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} + b\right) \tag{2.3}$$

This vector formulation serves as the basis for the matrix representation of an entire neural network layer, where multiple neurons are evaluated simultaneously.

2.3.2 Neural Networks

A neural network is a collection of interconnected neurons organized into layers. The input layer receives the feature vector, while the output layer produces the final prediction. The hidden layers between the input and output layers perform intermediate computations that allow the network to learn increasingly abstract representations of the input data [13]. The parameters of the network, including the weights and biases, are learned from data using optimization algorithms such as gradient descent [14]. The ability of neural networks to learn complex functions from data has made them a powerful tool for a wide range of applications in machine learning and artificial intelligence.

To understand the terminology: Artificial neural network is the broadest term, encompassing all brain-inspired, connectionist models. A feedforward neural network is a subcategory where information flows strictly in one direction (from input to output) without forming cycles, but its layers need not be fully connected [13]. The multilayer perceptron is a feedforward network with one or more fully connected (also called dense) hidden layers, where every neuron in one layer connects to every neuron in the next [13].

Multilayer Perceptron Network

The most common form of such architectures is the *Multi-Layer Perceptron (MLP)*, also referred to as a feedforward neural network [13]. In an MLP, where information flows in one direction from the input layer through one or more fully connected hidden layers to the output layer. In an MLP, each neuron in a layer receives the outputs of the neurons in the preceding layer as inputs. In a fully connected layer, every neuron is connected to every neuron in the subsequent layer [13]. The output of a layer therefore serves as the input to the next layer.

The computation of an entire layer can be expressed compactly using matrix notation. Let $\mathbf{x}^{(l-1)}$ denote the output vector of the previous layer, $\mathbf{W}^{(l)}$ the weight matrix, and $\mathbf{b}^{(l)}$ the bias vector of layer l. M are the number of neurons in layer l and N the number of neurons in the previous layer $l-1$. The pre-activation vector is given by:

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)}\mathbf{x}^{(l-1)} + \mathbf{b}^{(l)}, \tag{2.4}$$

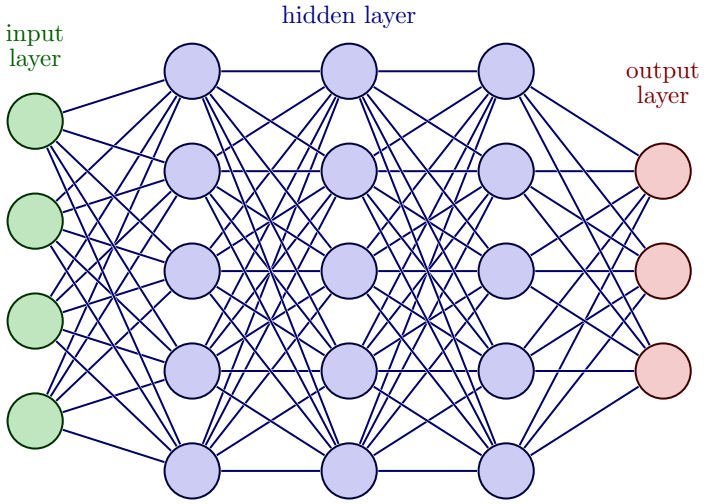


Figure 2.3: Neural Networks Architecture: Architecture of an Multilayer Perceptron Network. Circles represent perceptrons. Image source: [15].

which can be written explicitly as:

$$\begin{bmatrix} w_{11} & \cdots & w_{1N} \\ w_{21} & \cdots & w_{2N} \\ \vdots & \ddots & \vdots \\ w_{M1} & \cdots & w_{MN} \end{bmatrix}^{(l)} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_N \end{bmatrix}^{(l-1)} + \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_M \end{bmatrix}^{(l)} = \begin{bmatrix} w_{11}x_1 + \cdots + w_{1N}x_N + b_1 \\ w_{21}x_1 + \cdots + w_{2N}x_N + b_2 \\ \vdots \\ w_{M1}x_1 + \cdots + w_{MN}x_N + b_M \end{bmatrix} \tag{2.5}$$

In other words, each neuron in layer l computes its own weighted sum over all outputs of the previous layer and then adds its corresponding bias term.

Equivalently, the bias can be absorbed into the matrix multiplication by augmenting the input vector with a constant entry 1 and extending the weight matrix by one additional column:

$$\begin{bmatrix} z_1^{(l)} \\ z_2^{(l)} \\ \vdots \\ z_M^{(l)} \end{bmatrix} = \begin{bmatrix} w_{11}^{(l)} & \cdots & w_{1N}^{(l)} & b_1^{(l)} \\ w_{21}^{(l)} & \cdots & w_{2N}^{(l)} & b_2^{(l)} \\ \vdots & \ddots & \vdots & \vdots \\ w_{M1}^{(l)} & \cdots & w_{MN}^{(l)} & b_M^{(l)} \end{bmatrix} \begin{bmatrix} x_1^{(l-1)} \\ x_2^{(l-1)} \\ \vdots \\ x_N^{(l-1)} \\ 1 \end{bmatrix} \tag{2.6}$$

Then the activation function is applied element-wise to the resulting pre-activation vector $\mathbf{z}^{(l)}$ to produce the output of layer l:

$$\mathbf{x}^{(l)} = \phi(\mathbf{z}^{(l)}) = \phi\left(\mathbf{W}^{(l)}\mathbf{x}^{(l-1)} + \mathbf{b}^{(l)}\right). \tag{2.7}$$

Non-Linear Activation Functions in Neural Networks

The introduction of non-linear activation functions is essential, as a composition of purely linear transformations would again result in a linear model [13]. By combining

multiple layers with non-linear activations, neural networks can approximate highly complex functions and learn non-linear, intricate patterns from data [16]. In fact, sufficiently large neural networks are universal function approximators, meaning they can approximate a wide range of continuous functions to arbitrary accuracy. Common activation functions used in modern neural networks include the functions GELU (Gaussian Error Linear Unit) [17], ReLU (Rectified Linear Unit) [18], Leaky ReLU [19], the tanh (hyperbolic tangent) [20] and the Swish function [21]. But there exist many more. Each of these functions has its own characteristics and can result in different learning dynamics and performance.

Prediction

The output layer is responsible for producing the network's final prediction. Its activation function is chosen based on the task:

- **Regression:** For regression problems, a linear activation directly outputs a continuous value.
- **Binary Classification:** For binary classification (true/false), a single neuron with a sigmoid function outputs a probability between 0 and 1.
- **Multi-Class Classification:** For multi-class classification, a softmax layer produces a probability distribution over the classes, where each output neuron corresponds to a specific class and the softmax function ensures that the outputs sum to 1, representing the predicted probabilities for each class.

The choice of activation function in the output layer is crucial, as it directly affects the interpretation of the network's predictions and the appropriate loss function to use during training.

Loss Function

The loss function $\mathcal{L}$ is a measure of how well the network is able to predict the target value and quantifies the difference between the network's predictions and the true labels in the training data. The goal of the training process is to minimize the loss function by adjusting the weights of the network. The loss function can be any differentiable function, but common choices are Mean Squared Error (MSE) for regression tasks and cross-entropy for classification tasks.

- **Mean Squared Error (MSE) Loss Function:** Commonly used for regression tasks, where the goal is to predict continuous values. The Mean Squared Error (MSE) loss function is defined as:

$$\mathcal{L} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2 \tag{2.8}$$

  where y_i is the target value for the i-th sample, $\hat{y}_i$ is the predicted value for the i-th sample, and N is the number of samples.

- **Cross-Entropy Loss Function:** Commonly used for classification tasks, where the goal is to predict discrete class labels.
  - *Binary Classification:* For binary classification problems, the network typically uses a single output neuron with a sigmoid activation. The output $\hat{y}_i \in [0, 1]$ represents the predicted probability of the positive class. The binary cross-entropy loss is then defined as:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^{N} (y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)) \tag{2.9}$$

    where $y_i \in \{0, 1\}$ denotes the true label of the i-th sample, $\hat{y}_i$ is the predicted probability of the positive class, and N is the number of samples.
  - *Multi-Class Classification:* For multi-class classification problems with C classes, the network output is typically normalized using a softmax function to obtain a probability distribution over all classes (which sums up to 1 over the classes). The corresponding cross-entropy loss is defined as:

$$\mathcal{L} = -\frac{1}{N} \sum_{n=1}^{N} \sum_{i=1}^{C} y_{n,i} \log(\hat{y}_{n,i}) \tag{2.10}$$

    where $y_{n,i}$ represents the one-hot encoded ground-truth label and $\hat{y}_{n,i}$ is the predicted probability for class i of sample n.

2.3.3 Training Neural Networks

The parameters of a neural network, namely the weights and biases, are learned from data during training. This is typically achieved by minimizing a loss function $\mathcal{L}$ using optimization methods such as *gradient descent* and its variants. The gradients required for optimization are efficiently computed using the *backpropagation* algorithm, which propagates the prediction error from the output layer back through the network [14]. This learning process enables neural networks to adapt their internal representations and achieve strong performance across a wide range of machine learning tasks.

What is a Gradient?

The gradient is a fundamental concept in optimization and machine learning [14, 22]. It describes how a function changes with respect to its input variables and points in the direction of the steepest increase of the function. In the special case of a function with only one variable, the gradient reduces to the first derivative, which corresponds to the slope of the function at a given point. In simple terms, the gradient indicates how the function value changes when making a small step to the right or left away from a specific point.

To better understand the concept of gradients, consider Figure 2.4. We want to determine the slope of the blue curve at the point $x = 1$. To do this, we calculate the gradient at that point. First, we compute the derivative $f'(x) = 2x$ of the function

$f(x) = x^2 - 3$ and evaluate it at $x = 1$. The first derivative $f'(x)$ provides the slope at any point on the function $f(x)$. In this example, the slope at $x = 1$ is $f'(1) = 2$. To verify this result, the slope triangle is shown in Figure 2.4. It provides a visual representation of the slope and confirms the calculated value of with the slope $m = 2$.

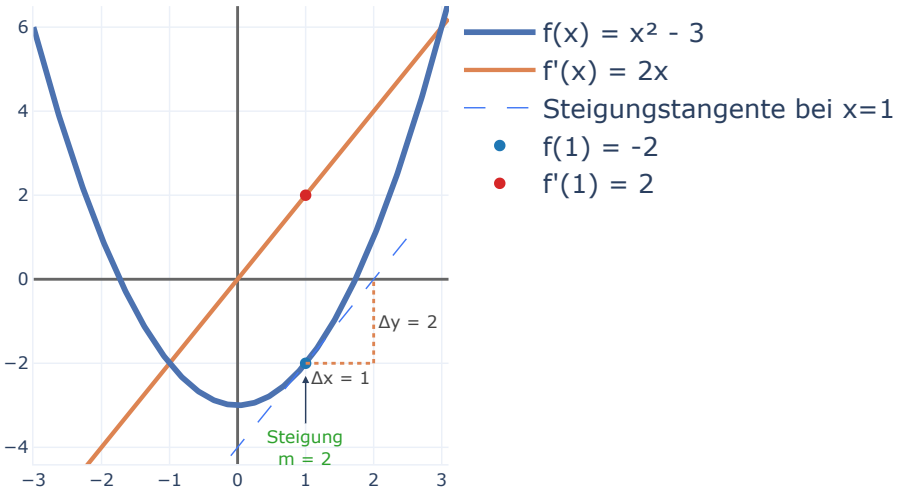


Figure 2.4: Understanding Gradients: The gradient of a 1D function $f(x)$ at point $x = 1$ is calculated with the derivative $f'(x)$. It represents the slope of the tangent line at that point.

In general, the gradient indicates whether a function is increasing or decreasing at a specific point and how steeply it changes. A positive gradient means that the function is increasing, while a negative gradient indicates that the function is decreasing. The magnitude of the gradient describes how steeply the function changes. Larger magnitudes correspond to steeper slopes, while values close to zero indicate relatively flat regions.

Gradient Descent and Parameter Update

Gradient descent is an iterative optimization algorithm that is used to minimize a loss function [22]. The basic principle of gradient descent can be understood by considering a simplified setting with a single parameter—a network weight—as illustrated in the Figure 2.5. The horizontal axis represents the value of the weight, while the vertical axis represents the corresponding loss, which quantifies the discrepancy between the predicted and true values of the training data.

The resulting curve is the loss landscape for that one parameter. At any point on the loss curve, the derivative (or gradient) indicates the local slope of the function. The

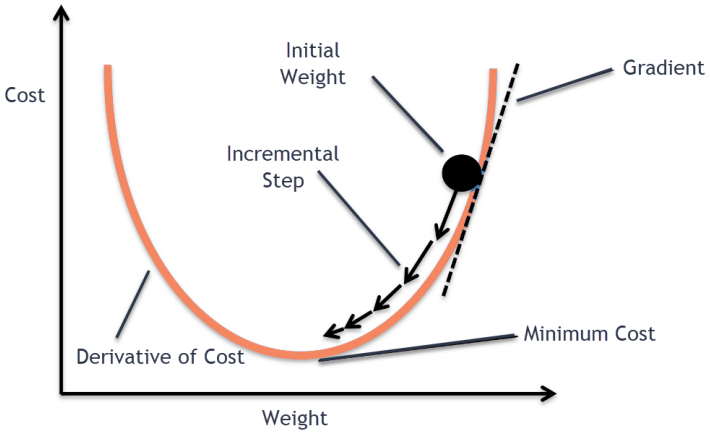


Figure 2.5: Neural Network: Simplified gradient descent based on one network weight. Image source: [23].

current value of the weight, represented by the black dot, must be adjusted such that the loss decreases. Gradient descent achieves this through the following two steps:

1. **Calculate the gradient:** Compute the gradient of the loss function with respect to the weight. The gradient indicates how the loss changes as the weight is adjusted.
2. **Update the weight:** The weight is then updated by moving in the opposite direction of the gradient. Since the gradient points towards the direction of steepest increase, moving against it results in a reduction of the loss. The basic parameter update is defined as:

$$\mathbf{W}_{t+1} = \mathbf{W}_t - \alpha \cdot \nabla \mathcal{L}(\mathbf{W}_t) \tag{2.11}$$

where α denotes the learning rate, which controls the step size of the update, and $\nabla \mathcal{L}(\mathbf{W}_t)$ is the gradient of the loss function with respect to the parameter at iteration t.

In the example shown in Figure 2.5, the gradient is positive. Consequently, increasing the weight would increase the loss. To reduce the loss, the weight must therefore be moved in the opposite direction (to the left). This procedure is repeated iteratively until the algorithm reaches a minimum of the loss function or another stopping criterion is satisfied. The successive parameter updates are illustrated by the black arrows in Figure 2.5. At the minimum, the gradient and the slope are zero and the parameter update becomes zero as well. The algorithm has converged.

Goal of Training Neural Networks: The goal is to find the optimal set of parameters that minimizes the loss function, which corresponds to the best fit of the model to the data, meaning the network's predictions are sufficiently accurate [22].

Optimization Problems

But not every minimum found is a good solution. The loss landscape of neural networks is highly non-convex and can contain many local minima, saddle points, and

flat regions, where the gradient is zero but the loss is not minimized. If the gradient gets zero, the optimization process get stuck [22]. Figure 2.6 shows a loss landscape with two parameters. The black dot represents the inital parameter values and the line shows the trajectory of the parameter updates during training. As seen it depends on the initial parameter values, whether the algorithm converges to the global minima, local minama or to the saddle point in the middle of the two hills.

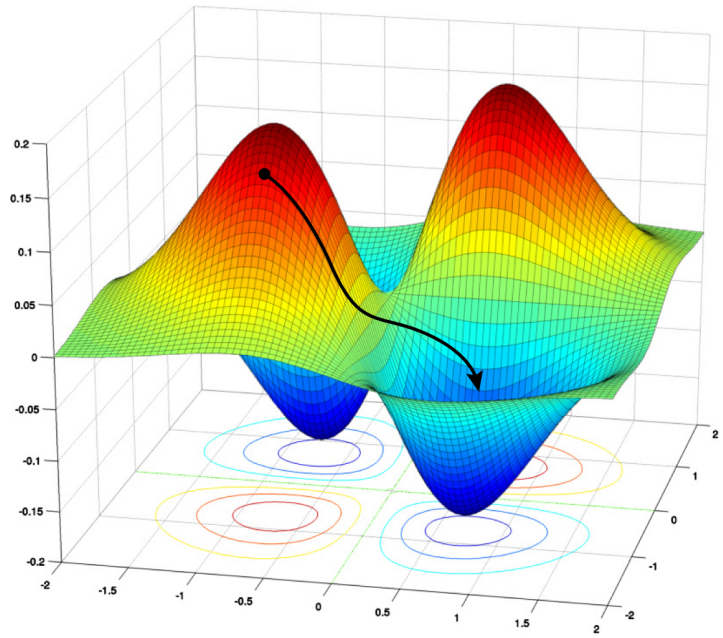


Figure 2.6: Neural Networks and their optimization Problems: The figure shows a loss landscape of two parameters, which has a local minima and a saddle point, where the training precess could get stuck. Image source: [24].

In the shown examples, the loss landscape was known, but in practice, the loss landscape of neural networks is typically unknown and can be highly complex. Neural networks often consists of millions of parameters and therefore they have a high-dimensional loss landscape. The Algorithm works in such high-dimensional spaces in the same way. Therefore, training neural networks can be challenging and may require careful tuning of hyperparameters such as the learning rate, as well as the use of techniques like momentum, adaptive learning rates or regularization to improve convergence and generalization performance [22]. For this many variants of the basic gradient descent algorithm have been developed, which is not the scope of this work.

Backpropagation

The gradients required for updating the network parameters in gradient descent are computed using the *backpropagation algorithm* [14]. Backpropagation efficiently propagates the prediction error from the output layer back through the network and computes the gradients of the loss function $\mathcal{L}$ with respect to all weights $\mathbf{W}$ in the model. Based on these gradients, each parameter w_{ij}^{l} is adjusted proportionally to its contribution to the overall error.

More precisely, backpropagation computes the partial derivatives of the loss function with respect to all trainable parameters in the network. These gradients quantify how sensitive the loss is to small changes in each parameter and therefore indicate how each parameter should be adjusted to reduce the overall error. Parameters that contribute more strongly to the error receive larger updates, while less influential parameters are adjusted more mildly. The training process for a neural network can therefore be summarized by the following update rule:

$$\mathbf{W}_{t+1} = \mathbf{W}_t - \alpha \cdot \nabla_{\mathbf{W}_t} \left(\frac{1}{N} \sum_{i=1}^{N} \mathcal{L}\big(\mathbf{y}_i, f_{\mathbf{W}_t}(\mathbf{x}_i)\big) \right) \tag{2.12}$$

where $f_{\mathbf{W}}$ denotes the neural network with parameters $\mathbf{W}$, $\mathcal{L}$ is the loss function measuring the difference between the predicted output $f_{\mathbf{W}}(\mathbf{x}_i)$ and the target value $\mathbf{y}_i$, α is the learning rate, N is the number of training samples, and $\nabla_{\mathbf{W}_t}$ represents the gradient of the loss function with respect to the weights at iteration t.

By iteratively applying this update rule, the network parameters are gradually adjusted in a direction that reduces the loss function. This iterative optimization process is what allows the neural network to learn meaningful representations from training data and improve its predictions over time.

Besides the standard gradient descent algorithm, which computes the gradient over the entire dataset, several more efficient variants are commonly used in practice. In stochastic gradient descent (SGD), the exact gradient is not evaluated on the full dataset [25, 26]. Instead, it is approximated using only a single randomly selected training sample at each iteration. A further extension is mini-batch gradient descent, where the gradient is computed on a small subset of the training data (a mini-batch) rather than on a single sample or the full dataset [13, 22]. This approach reduces the computational cost per update while still providing a more stable gradient estimate compared to pure stochastic gradient descent.

2.3.4 Evaluation

Evaluation Metrics

To measure the performance of a machine learning model, various evaluation metrics can be used depending on the specific task and problem setting. These metrics provide quantitative measures of how well a model performs and allow comparisons between different models or configurations. While common examples include accuracy, precision, recall, and the F1-score for classification tasks, as well as mean squared error for regression tasks. However, a detailed discussion of these metrics is beyond the scope of this work.

Evaluation

The performance of a machine learning model is typically evaluated using data that has not been seen during training, referred to as test data. This allows an unbiased

assessment of the model's ability to generalize to new, unseen samples. There exist in general three main scenarios regarding the model's performance on training and test data:

- **Underfitting:** A model that underfits performs poorly on both training and test data. It fails to capture the underlying patterns in the training data, resulting in high error on both datasets.
- **Overfitting:** An overfitted model achieves high performance on the training data but fails to generalize well to the test data. It has essentially memorized the training samples, including noise and outliers, which leads to poor performance on new data.
- **Well-Balanced Model:** A well-balanced model achieves consistently good performance on both training and test datasets. It captures the underlying patterns in the training data without overfitting, allowing it to generalize effectively to unseen samples.

2.4 Convolutional Neural Networks

A Convolutional Neural Network (CNN) is a deep neural network architecture designed to process grid-like data, especially images [27, 28]. CNNs are particularly effective for visual tasks because they exploit the spatial structure of an image and learn local patterns such as edges, corners, and textures. Unlike fully connected networks, CNNs do not treat every pixel independently. Instead, they use small learnable filters that are applied across the image to detect relevant structures at different positions [29–31].

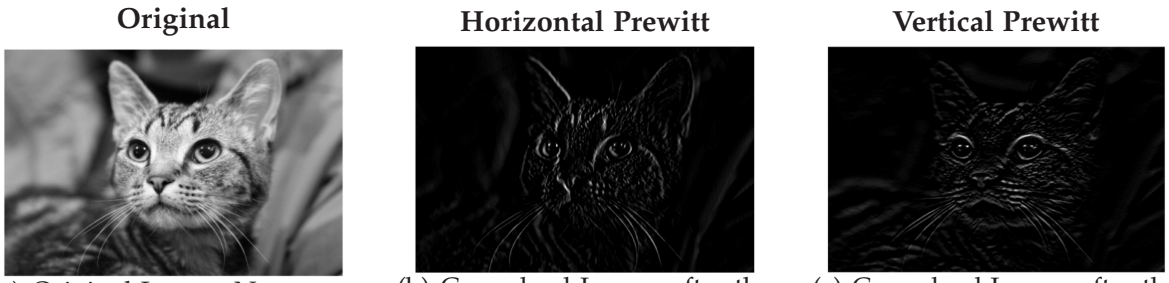
Convolution

The main idea of a CNN is the convolution operation, where a filter, also known as kernel, is applied to the image to extract features. For example, the Prewitt kernels, also known as edge detection filters, can be used to detect horizontal and vertical edges in an image:

$$K_{\text{horizontal}} = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix} \qquad K_{\text{vertical}} = \begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix} \tag{2.13}$$

Figure 2.7 shows the result, after applying the Prewitt kernels to a grey-scale image. There exists a wide variety of predefined filters to extract various of visual structures from the image.

A Filter or kernel is defined as a small matrix of weights, is moved across the image and element-wise multiplied with the underlying pixel values [29–31, 33]. This is possible, because an image is technically a matrix of pixel values too. The resulting values are stored in a new matrix called a feature map, which highlights the patterns



(a) Original Image: No convolution is applied

(b) Convolved Image, after the horizontal Prewitt kernel is applied to the original image

(c) Convolved Image, after the vertical Prewitt kernel is applied to the original image

Figure 2.7: Image-based instructions for task execution. Image sources: [32].

detected by the kernel. Since the same kernel is reused at every pixel location, CNNs are able to detect the same feature independent of its position in the image. Figure Figure 2.8 illustrates the mathematical convolution operation, where a 3×3 kernel is applied to a 6×6 image. The results are stored in a 3×3 matrix.

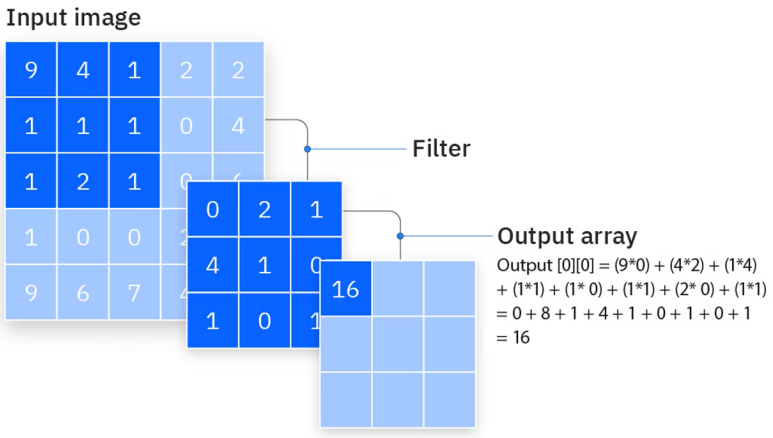


Figure 2.8: CNN Convolution Introduction. Image source: [34].

Figure Figure 2.9 shows how the kernel (dark blue) is slided across the image (light blue) step by step. At each location the element-wise products are summed up to obtain one output value, which is stored in a feature map (olive green).

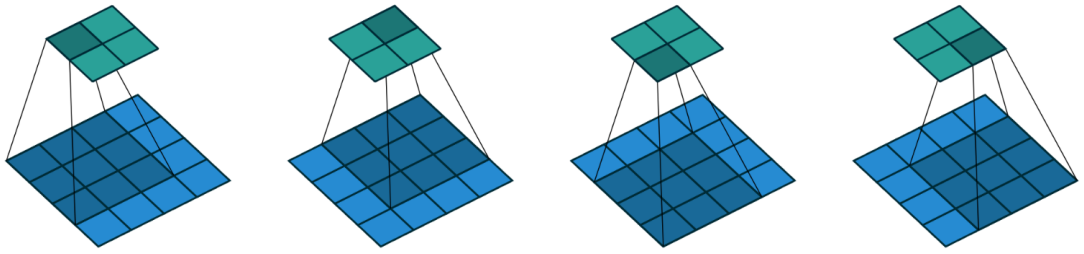


Figure 2.9: CNN Convolution Step-by-Step. Image source: [33].

Figures Figure 2.10 shows an alternative view of the convolution process.

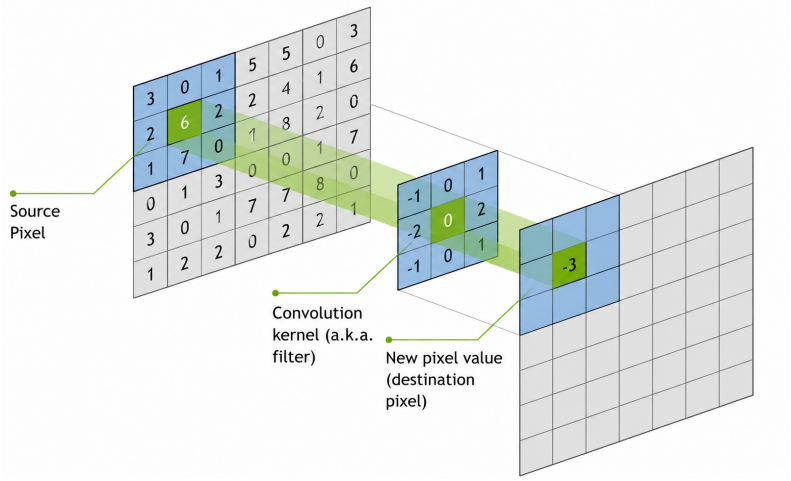


Figure 2.10: CNN Convolution Introduction. Image adapted from: [35].

Padding

As seen in the Figures 2.8, 2.9 and 2.10, the feature map is smaller than the original image, because of the convolution operation. To preserve the spatial resolution of the input image, padding is often added around the border before applying the convolution, see Figure 2.11 [29–31, 33]. With zero-padding, the image size can remain unchanged after the convolution, which is useful when multiple convolution layers are stacked.

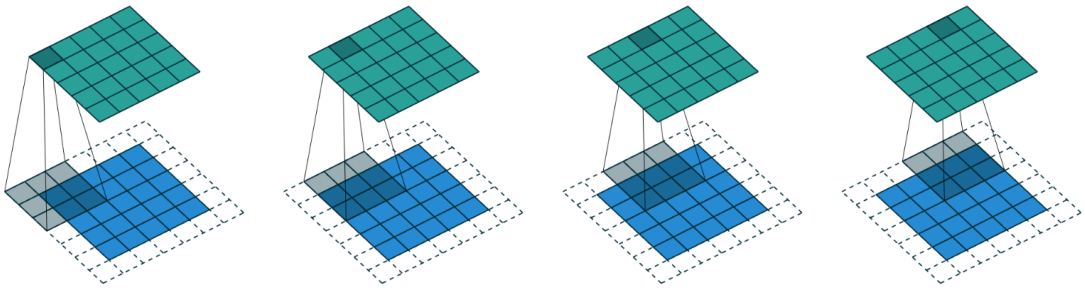


Figure 2.11: CNN Convolution with Padding. Image source: [33].

A CNN consist of multiple kernels, which are applied in parallel and in multiple layers to the image and the resulting feature maps. Importantly, in a CNN the kernels are not fixed manually, instead they are learned during training, allowing the network to find the optimal filters for the task success [29–31].

Activation Function

After the convolution operation, a non-linear activation function is applied to the resulting feature map [29–31]. Activation functions introduce non-linearity into the network, allowing CNNs to learn complex visual patterns that cannot be represented by linear transformations alone [36].

The most commonly used activation function is the Rectified Linear Unit (ReLU) [29, 31, 36]: which replaces all negative values with zero while leaving positive values unchanged:

$$f(x) = \max(0, x) \tag{2.14}$$

By applying the ReLU function after each convolution, the network can learn increasingly complex feature representations across multiple layers.

Pooling Layer

Another central component of CNNs is the pooling layer [29, 31]. Pooling reduces the spatial size of the feature maps while retaining the most important information. This lowers the computational cost and increases robustness to small shifts in the input image.

A common choice is a 2×2 pooling window with a stride of 2. In this case, the feature map is divided into non-overlapping 2×2 regions, and each region is summarized by a single value [29, 30]. As a result, the width and height of the feature map are reduced by a factor of two.

Two common pooling methods are max pooling and average pooling [29, 30], as illustrated in Figure 2.13. In max pooling, the largest value within each pooling region is selected and propagated to the output feature map. In average pooling, the mean value of all elements within the region is computed. Both methods reduce the spatial resolution while preserving the most relevant information contained in the feature map.

Pooling layers are typically applied after convolution layers and allow the network to build increasingly compact feature representations. While early convolution layers extract local patterns such as edges and textures, pooling helps summarize these features and enables deeper layers to focus on higher-level visual structures [29–31].

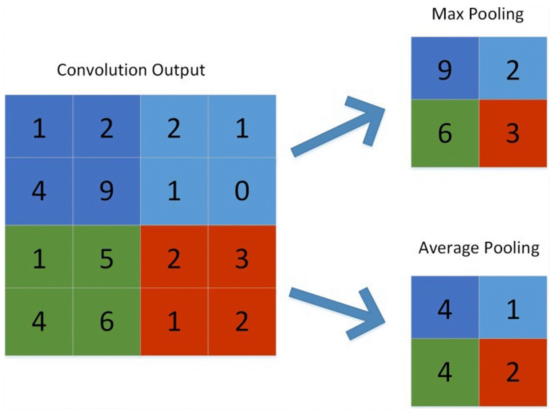


Figure 2.12: CNN Pooling. Image source: [37].

CNN Pipeline

The common CNN processing sequence is: (1) convolution, (2) activation function, and (3) pooling, while the activation function is often not mentioned explicitly. A CNN

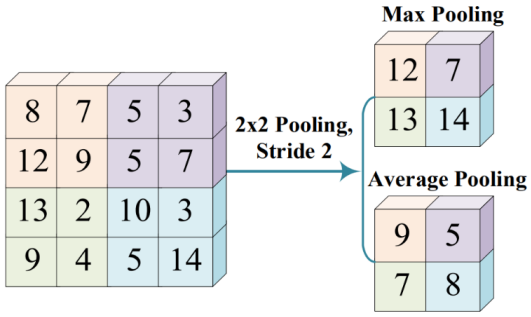


Figure 2.13: CNN Pooling. Image source: [29].

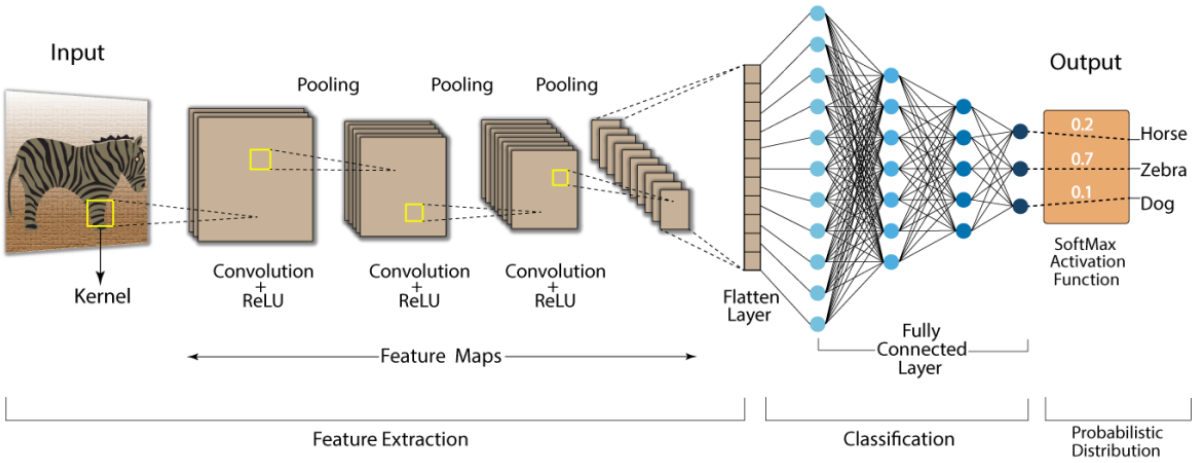


Figure 2.14: CNN Pipeline. Image source: [38].

usually consists of several repeated blocks of convolution and pooling layers. Early layers detect simple features such as edges and corners, while deeper layers combine these basic features into more complex structures. After several convolution and pooling stages, the feature maps are flattened into one big vector and passed through a MLP or a classification head in a classification task to produce the final predictions, see Figure 2.14. Through backprobagating the prediction error, the networks parameter are adjusted during training.

RGB Images

The same principle can be extended to RGB images [28, 29, 31]. While a grayscale image contains only a single channel, an RGB image consists of three color channels representing red, green, and blue values. Consequently, a convolution kernel must span all input channels. For a kernel size of 3×3, a common way is to use an $3 \times 3 \times 3$ kernel for an RGB image [28, 29, 31]. This kernel is applied across all three color channels simultaneously, and the resulting values are summed up to produce a single output value in the feature map. By learning appropriate kernels during training, CNNs can effectively capture complex visual patterns across all color channels in RGB images.

2.5 Transformer

A Transformer is a deep neural network architecture designed to process sequential data, especially text [39]. It's the foundation for most modern LLMs, enabling them to process and understand language effectively. Unlike earlier architectures, which process tokens one after another, Transformers compare all tokens in a sequence in parallel and learn which parts of the input are relevant to each other. This is done through the attention mechanism and was the core innovation of the Transformer architecture [39–42]. The main idea is simple: when the model reads one token, it does not treat all other tokens equally. It assigns higher importance to the tokens that are more relevant in the current context. This makes Transformers especially powerful for language tasks, because meaning often depends on long-range dependencies in the sentence.

Architecture Overview

The original Transformer architecture was introduced by Vaswani *et al.* [39] and consists of two parts: the encoder and the decoder. See Figure 2.15 for an overview.

1. **Encoder:** The encoder reads the full input sequence and converts it into a contextual latent representation. Each token is transformed not only based on its own identity, but also based on the other tokens in the sequence. In this way, the encoder builds a rich internal representation of the input.
2. **Decoder:** The decoder generates the output sequence autoregressively, meaning one token at a time. To produce the next token in the output sequence, it uses at each prediction step: (1) The contextual representation from the encoder; (2) The previously generated output tokens from the decoder. This allows the decoder to produce the next token while taking the source sequence into account.

While the examples in this section are based on text processing to facilitate understanding of the Transformer architecture, the underlying principles generalize to other modalities, such as images or videos.

2.5.1 Input Embeddings

Before a Transformer can process data, the data must be converted into a numerical representation. This is the role of the input embedding layer [42]. Given an input text such as "Data visualization empowers users to", the model first tokenizes the text, then maps each token to a vector representation, and finally adds positional information [41, 42]. The resulting embedding combines semantic meaning with information about token order.

1. **Tokenization:** The input text is split into smaller units called tokens. These tokens can be a word, a subword or a single character. For example, the sentence might be tokenized into ["Data", "visualization", "empowers", "users", "to"]. Each token is assigned a unique token ID.

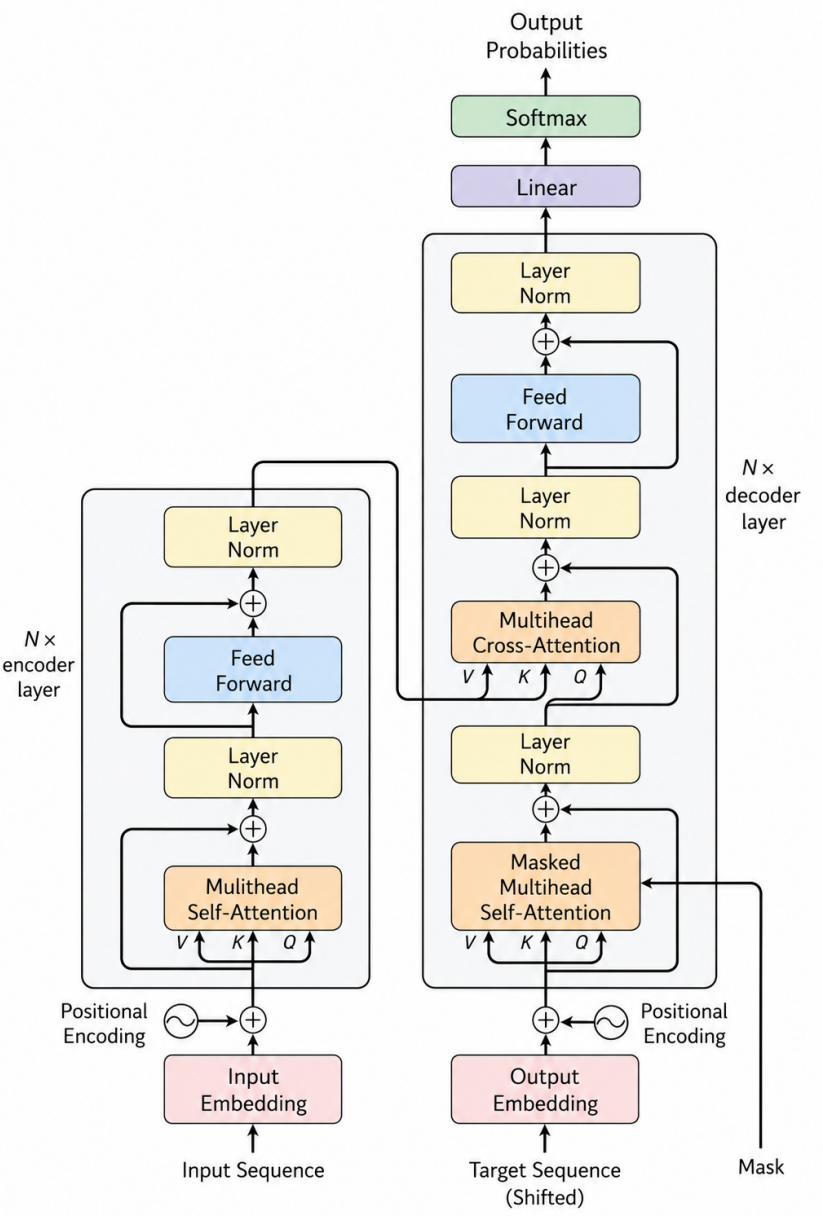


Figure 2.15: Transformer Architecture. Image based on: [39].

2. **Embedding:** Each token is then mapped one-to-one to a high-dimensional vector representation. This embedding captures the semantic meaning of the token. For example, the token "Data" might be represented as a 768-dimensional vector in the embedding space.

3. **Positional Encoding:** Since the Transformer does not process tokens sequentially, it needs an explicit way to represent token order. Positional encodings provide this information. Depending on the model, these encodings may be fixed or learned. They are added to the token embeddings so that the model can distinguish between, for example, “dog bites man” and “man bites dog”.

4. **Final Embedding:** The final input representation is obtained by combining token

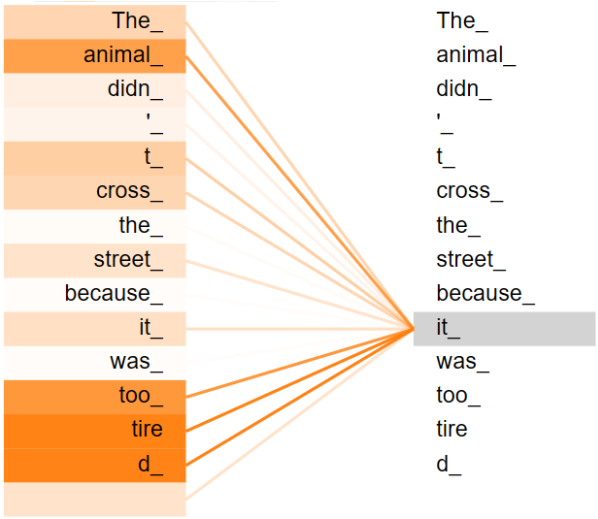


Figure 2.16: Attention Weights of the word "it" in a sentence. Image source: [43]

embeddings and positional information, typically through element-wise addition. This representation is then passed into the Transformer layers.

2.5.2 Attention Mechanism

The Attention mechanism is responsible for capturing contextual relationships between tokens by directly computing their relevance to one another. This allows the model to understand long-range dependencies and contextual relationships of the tokens, leading to more coherent and relevant output [40–42]. In Figure 2.16 the attention weights of the word "it" in the sentence "The animal didn't cross the street because it was too tired" are visualized, after the input is processed through the Encoder attention mechanism. The attention weights allows the model to decide which other tokens in the sequence are most relevant.

Compute Attention Weights. In the self-attention mechanism, each token embedding vector $\mathbf{x}^{(i)}$ is transformed into three different representations: query $\mathbf{q}^{(i)}$, key $\mathbf{k}^{(i)}$ and value $\mathbf{v}^{(i)}$. These representations are obtained through learnable linear transformations with parameters (W_q, W_k, W_v) that are optimized during training [40–42].

- **Query $\mathbf{q}^{(i)} = \mathbf{x}^{(i)}\mathbf{W}_q$:** The query vector represents the token for which we want to compute attention. It is used to determine how much attention this token should pay to other tokens in the sequence.
- **Key $\mathbf{k}^{(i)} = \mathbf{x}^{(i)}\mathbf{W}_k$:** The key vector represents the token against which the query is compared. It is used to compute the relevance score between the query and the token.
- **Value $\mathbf{v}^{(i)} = \mathbf{x}^{(i)}\mathbf{W}_v$:** The value vector contains the actual information of the token. It is weighted by the attention scores to produce the final output of the self-attention mechanism.

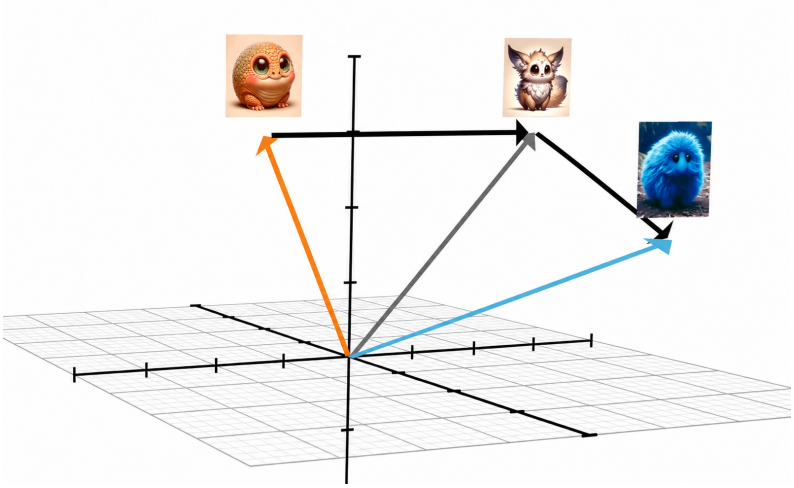


Figure 2.17: Embedding Space of the Value Vectors: The Vector of "creature" is shifted in the Embedding space, based on the context. Image source: [40].

Single-Head Self-Attention

The query vectors are then compared with the key vectors of all tokens in the sequence using the dot product (scalar product), producing relevance scores that indicate how strongly tokens are related to one another. To stabilize training, these scores are scaled by the square root of the key dimension d_k and normalized using a softmax function, resulting in attention weights. Finally, the attention weights are applied to the corresponding value vectors $\mathbf{v}^{(i)}$ and these weighted values are aggregated to produce the new context vector $\mathbf{z}^{(i)}$ and the output of the self-attention layer [40–42], see Figure 2.18.

$$\mathbf{z}^{(i)} = \text{softmax}\left(\frac{\mathbf{q}^{(i)} \cdot \mathbf{k}^{(i)T}}{\sqrt{d_k}}\right) \cdot \mathbf{v}^{(i)} \tag{2.15}$$

In this way, each token can incorporate information from the most relevant tokens in the sequence, enabling the model to capture contextual dependencies. But what does this mean technically? The value vector $\mathbf{v}^{(i)}$ represents the token's information in a semantically learned high-dimensional embedding space. Through the attention mechanism, the vector's direction and magnitude in this embedding space are adjusted, resulting in a different semantic meaning. In Figure 2.17, it is shown how the vector of the word "creature" is shifted in the embedding space based on the context "a fluffy blue creature". This illustrates how the attention mechanism allows the model to dynamically adjust the meaning of tokens based on their relationships with other tokens in the sequence [40].

Instead of computing attention separately for each token, the Transformer performs the computation for the entire sequence using matrix operations [42], see Figure 2.19. The matrix product $\mathbf{QK}^T$ computes the pairwise similarity scores between all tokens in the sequence in parallel, resulting in the attention score matrix.

$$\mathbf{Z} = \mathit{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{QK}^T}{\sqrt{d_k}}\right)\mathbf{V} \tag{2.16}$$

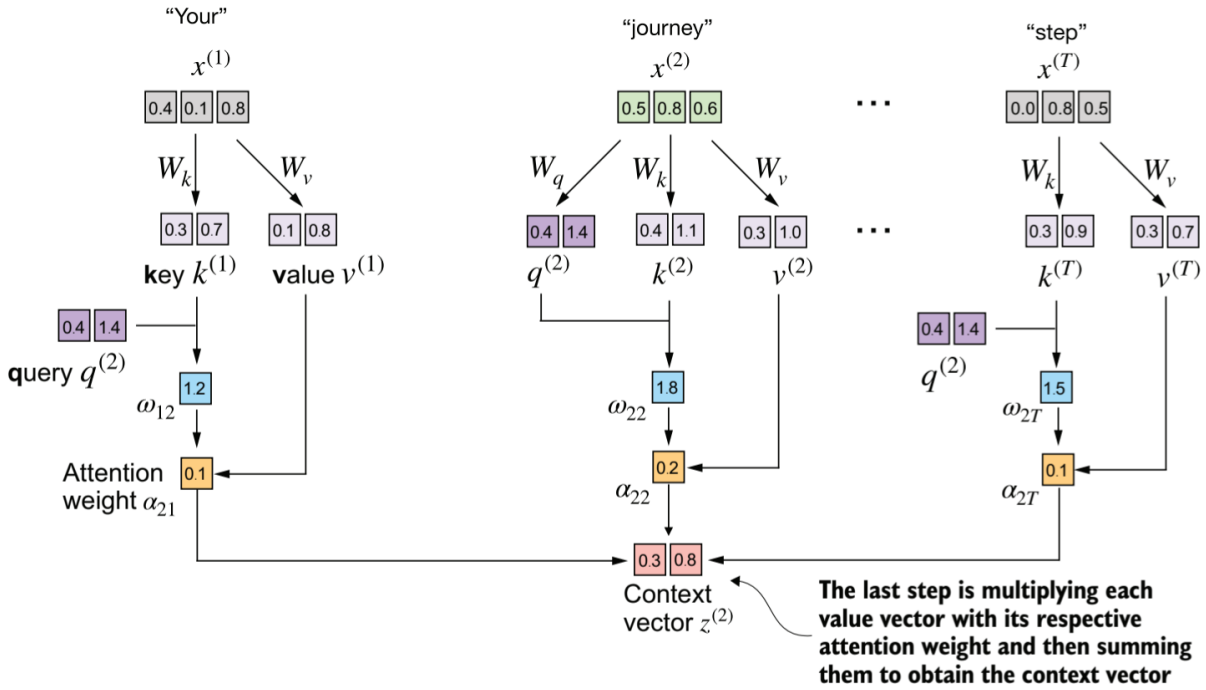


Figure 2.18: Self-attention: Compute the new context vector of each input embedding vector by combining the (attention)-weighted value vectors. Image source: [42].

Multi-Head Self-Attention

While a single attention mechanism can capture relationships between tokens, different types of dependencies may exist simultaneously within a sequence. To address this, the Transformer employs multi-head attention, which consists of several attention heads operating in parallel [39, 42]. Each head learns its own query, key, and value projections and therefore focuses on different aspects of the input data. For example, one head may capture syntactic relationships, while another focuses on semantic dependencies. The outputs of all heads are concatenated and projected through a linear layer to form the final representation, see Figure 2.20. This increases the model's ability to learn diverse linguistic patterns and contextual relationships. The number of heads is chosen such that the input embedding dimension is split across the heads. For example, if the input embedding dimension is 768 and there are 12 heads, each head would operate on a subspace of 64 dimensions. Otherwise, the output dimensions will not match the input dimensions. Technically the Query, Key and Value matrices of each head are combined computed within a larger matrix multiplication and then splitted up into the individual parts for each head, that every head can produce its own attention matrix.

2.5.3 Encoder Block

After the self-attention computation, the resulting representations are passed through a layer normalization step, followed by a MLP Network. A second layer normalization is applied after this transformation. This sequence of operations forms a single Transformer block, which can be stacked multiple times to increase model capacity

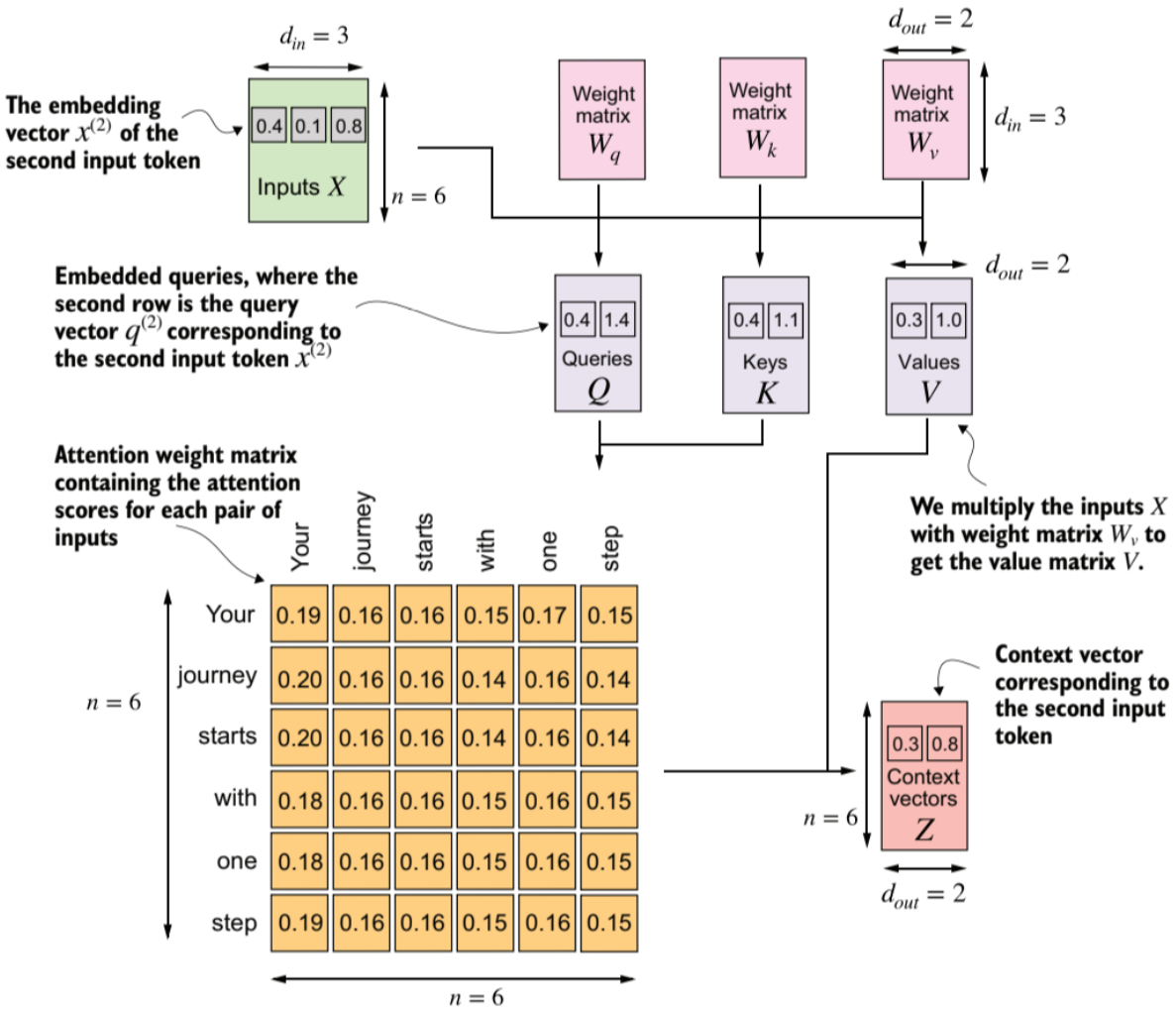


Figure 2.19: Single self-attention Block. Image source: [42].

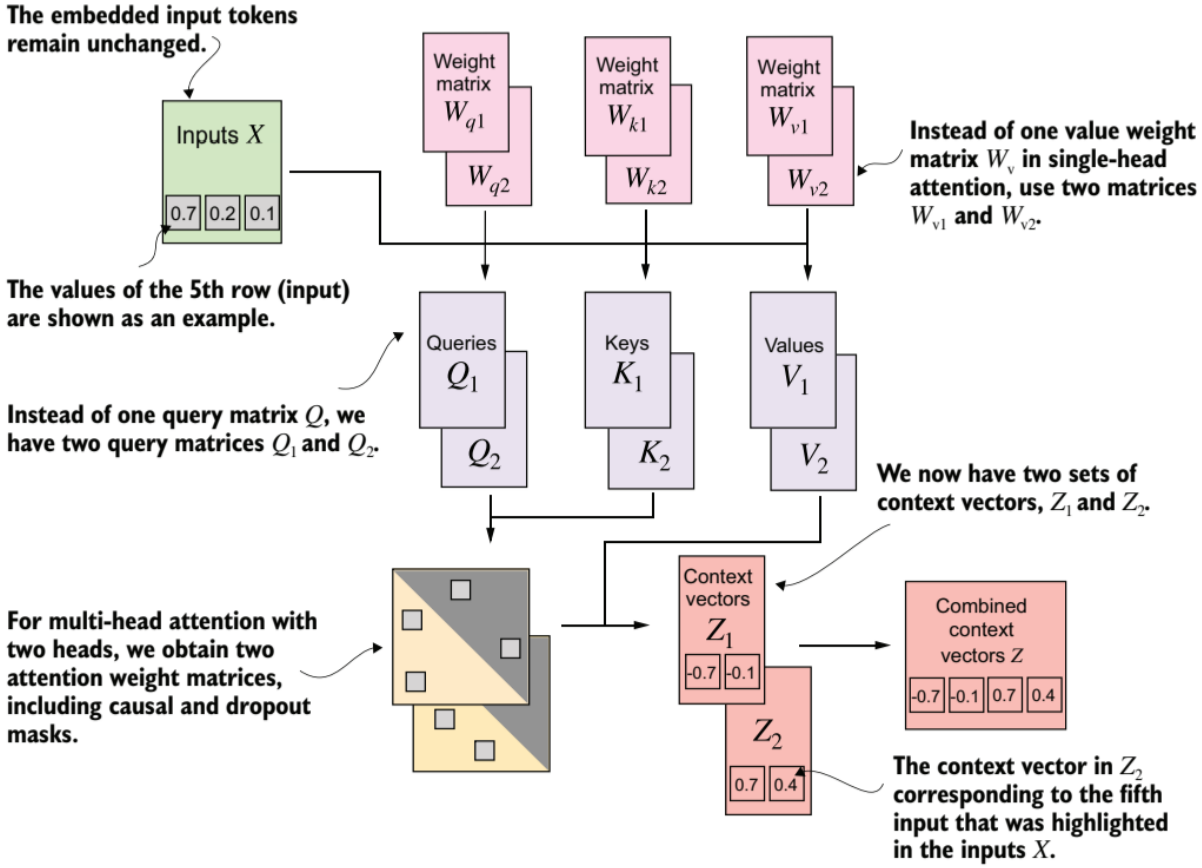


Figure 2.20: Multi-Head Attention Block. Image source: [42].

[39].

The MLP operates independently on each token representation. In contrast to the attention mechanism, which is responsible for exchanging information between tokens, the MLP refines and transforms each token embedding individually. It typically consists of two linear transformations with a non-linear activation function in between, allowing the model to learn more complex feature transformations.

A key component of the Transformer architecture is the use of residual connections, which are applied around both the attention block and the feed forward network [39]. These are shortcut connections that add the input of a sub-layer directly to its output. This mechanism improves gradient flow during backpropagation and mitigates the vanishing gradient problem, which is especially important in deep networks composed of many stacked Transformer blocks.

The output of the final encoder layer is a sequence of contextualized vectors, where each vector corresponds to one input token enriched with information from the entire sequence. This representation can then be used for downstream tasks such as classification, regression, or sequence generation.

2.5.4 Decoder Block

The decoder receives an input sequence that is first transformed using the same embedding procedure as the encoder (token embeddings + positional information) [39]. During generation, this input typically consists of previously generated output tokens. The embedded sequence is then processed through two distinct attention mechanisms: masked multi-head self-attention and cross-attention.

Masked Multi-Head Attention

Masked multi-head self-attention operates on the decoder's own input sequence, but with a causal mask that prevents each position from attending to any future positions [42]. The causal mask ensures that future tokens receive an attention weight of zero, making them invisible to the current position and ensures that the prediction for the next token depends only on previously generated tokens.

Cross-Attention

In cross-attention, the decoder's hidden states from the masked attention layer serve as queries Q, while the encoder's output serve as keys K and values V, allowing the decoder to condition its predictions on the encoded source sequence [39].

After these attention layers, the resulting representations are passed through a linear projection head followed by a softmax function, producing a probability distribution over the set of possible output elements [39, 41, 42]. The element with the highest probability is selected and appended to the generated sequence. This process repeats autoregressively until a special end-of-sequence symbol is produced. In our text

example, the vocabulary token with the highest probability is selected and appended to the generated sequence.

2.6 Vision Transformer

The Vision Transformer (ViT) [44] extends the Transformer architecture, originally developed for natural language processing, to computer vision tasks by applying the self-attention mechanism to image patches instead of words. In computer vision, attention enables a model to focus on relevant regions of an image and to capture and understanding relationships between different areas of an image. This allows the model to identify important objects and effectively model spatial dependencies across the entire image.

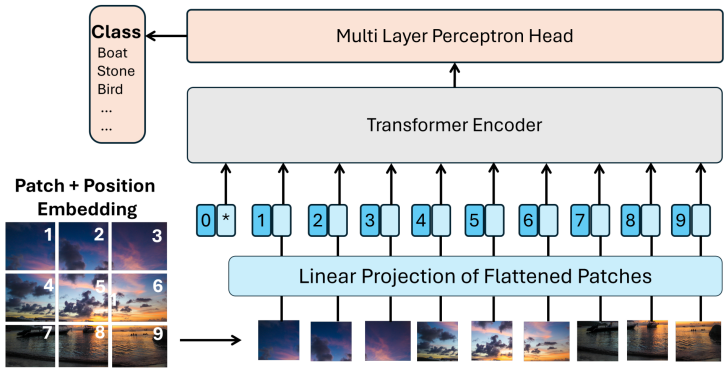


Figure 2.21: Vision Transformer: Overview of the architecture. Image source [45].

To process an image, ViT first divides the input image into a sequence of fixed-size, non-overlapping patches, see Figure 2.21. Each patch is flattened and linearly projected into a fixed-dimensional embedding vector, analogous to token embeddings in natural language processing.

Since the Transformer architecture does not inherently encode spatial information between the image patches, learnable positional embeddings are added to the patch embeddings. In addition, a learnable classification token (CLS token), denoted by an asterisk (*) in Figure 2.21, is prepended to the sequence and serves as a global representation of the whole image. During the attention computation, the CLS token interacts with all patch embeddings and gradually aggregates information from the entire image.

The resulting sequence is processed by a standard Transformer encoder consisting of multi-head self-attention and feed-forward layers. Through self-attention, the model learns global relationships between image patches and can capture long-range spatial dependencies that are difficult to model with traditional CNN approaches.

After the final encoder layer, the output representation of the CLS token serves as a global representation of the image and is passed through a multi-layer perceptron (MLP) head to generate the final prediction, such as class probabilities for image classification tasks.

Vision Transformers have demonstrated competitive performance on large-scale image classification benchmarks such as ImageNet [46], highlighting the effectiveness of attention-based architectures for visual representation learning [44]. In imitation learning, Vision Transformers can be used as visual encoders that extract global scene representations from camera observations, which are subsequently used by the policy network to predict actions.

2.7 Foundation Model & Multi-Model Model

The terms *Foundation Model* and *Multi-Modal Model* are often used in the context of modern AI systems, but they refer to different concepts. A foundation model is a large-scale model that is pre-trained on a wide variety of data and can be adapted to many downstream tasks, while a multi-modal model is designed to process and relate information from multiple data modalities, such as text, images, and audio, see Figure 2.22.

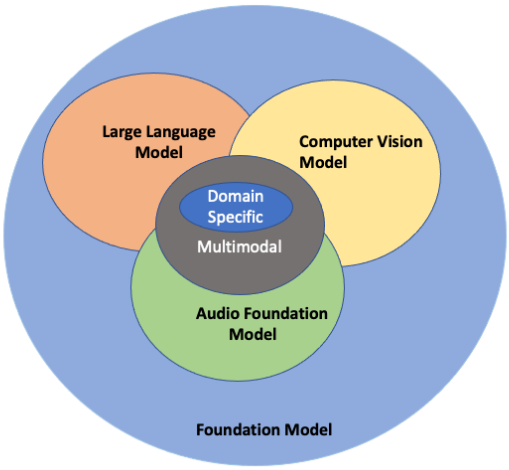


Figure 2.22: Foundation Model vs. Multi-Model Model. Image source: [47]

2.7.1 Foundation Model

Recent advances in artificial intelligence have been largely driven by the development of foundation models. A foundation model is a large-scale model that is pre-trained on vast amounts of data and can subsequently be adapted to a wide range of downstream tasks [48]. Instead of being designed for a single application, foundation models learn general patterns, concepts, and representations that can be reused across different domains and tasks.

A key characteristic of foundation models is their ability to generalize to problems that were not explicitly specified during training. This broad knowledge enables them to serve as a common basis for many specialized applications. Unimodal foundation models (e.g., GPT-3 for text [49], Stable Diffusion for images [50]) learn patterns within a single data type, whereas Multi-modal foundation models (e.g., CLIP [51], GPT-4V

[52], Gemini [53, 54]) learn joint representations across modalities such as text, images, and audio. Due to their versatility and strong generalization capabilities, foundation models have become the foundation of many modern AI systems.

2.7.2 Multi-Modal Model

A multimodal model is capable of processing and relating information from two or more data modalities, such as text, images, audio, or sensor measurements [55]. Unlike unimodal models, which operate on only a single type of data, multimodal models learn a shared representation that enables information from different modalities to be interpreted jointly. The integration of multiple modalities allows a model to leverage complementary information, handle missing data from one source by relying on another, provide more comprehensive insights, and improves model generalization [56].

For example, a multimodal model can analyze an image together with a textual instruction and use information from both inputs to answer a question or perform a task. This capability allows the model to connect visual observations with linguistic concepts and reason across different sources of information. Models such as CLIP align images and text within a common representation space [51], while more recent models such as Gemini can process as well as generate information across multiple modalities [53, 54].

Although many multimodal models can generate outputs in different modalities, such as images from text prompts, the defining characteristic of a multimodal model is its ability to understand and relate multiple forms of data rather than the specific modality of its output [52–54].

Multimodal Learning is a own learning paradigm in the field of deep learning and refers to the use of various data modalities, such as text, audio, sensor signals, and images/videos within a single learning concept at the same time [55, 57].

Vision-Language-Action Models

Vision-Language-Action (VLA) models extend multimodal models by incorporating robot actions as an additional output modality and can be ordered in the *Domain specific* section in Figure 2.22. While conventional multimodal models typically generate textual or visual outputs, VLA models directly generate robot actions that can be executed by a robotic. They combine visual observations from cameras with natural language instructions and translate this information into control commands for the robot. They often uses pretrainied foundation models for vision and language, that serves as backbones to provide world understanding. By jointly reasoning over vision, language, and actions, VLA models provide an end-to-end framework that enables robots to understand their environment, interpret user instructions, and autonomously perform complex manipulation tasks.

2.8 Reinforcement Learning

RL is both a fundamental discipline of ML and a distinct approach within Robot Learning. The robot learns a policy through trial and error using rewards provided from the given task and the environment. The actions performed by the agent are evaluated with a reward score, and this score is then used to gradually optimize the policy. See also figure 2.1 for a general graphical overview. Below are the key terms in RL explained in relation to robotics:

- **Agent:** The agent is the entity that learns to perform a specific behavior through interaction with its environment. In the context of this work, the agent corresponds to the robot, which has to execute a given action or task.
- **Environment:** The environment is the setting in which the agent/the robot operates. In other words, the environment is the robot's workplace, where it is deployed. In addition to the robot itself, it includes all relevant objects with which the robot interacts. The environment can be either the real world or a simulated environment in which the robot interacts during training or inference.
- **Robot Actions:** The Robot actions are simply the control commands the robot can send to its actuators, e.g. open or close the gripper, move the arm to position X, Y, Z, or the joint angles of the individual robot axes, etc.
- **Reward:** The reward is a numerical value that indicates how well the robot is performing in relation to the task objective. For example, if the robot successfully picks up an object, it might receive a positive reward, while if it fails or drops the object, it might receive a negative reward. The goal of Reinforcement Learning is to learn a policy that maximizes the cumulative reward over time.
- **Robot State:** The State is a complete description of the situation in the environment at a particular point in time. The robot state represents the current situation of the robot and its environment. It can include various types of information, such as the robot's position, orientation, joint angles, sensor readings, and even visual input from cameras [58]. The state is crucial for the robot to make informed decisions about which actions to take.
- **Observation:** An observation is the information that the robot receives from its sensors at a given time step. It may not provide a complete picture of the environment, and thus may not fully represent the true robot state. In our case, the observation is mostly equivalent to the state [58].
- **Policy:** The policy can be interpreted as the brain of the agent. The policy determines which action should be taken next when the agent is in a particular state in order to achieve the task objective. The Policy controls the robot's motion and actions.

  Formally, a policy can be represented as

$$\pi_\theta(a \mid s) \tag{2.17}$$

which defines a probability distribution over actions a, conditioned on the current state s. In other words, the policy specifies how likely the agent is to select a particular action given its current observation of the environment. In the classical meaning, the action a corresponds for example to a robot command, such as moving the gripper to a target position (x, y, z) or setting a joint angle to a specific value with a defined velocity. The state s represents the agent's observation, which may include, for example, a sequence of recent camera images (e.g., the last four frames) as well as additional sensor inputs such as proprioceptive signals.

Mathematical description of the RL problem:

The RL modularities are represented as follows:

- Action: a
- Reward: r
- State: s
- Observation: o
- Policy: π
- Time step: t, e.g. a_t is the action taken at time step t, a_{t+1} is the action taken at the next time step, etc.
- s': The next state after taking action a in state s, i.e. $s' = s_{t+1}$, $s = s_t$.

2.8.1 Markov Decision Process

RL is based on the Markov decision process (MDP). A MDP is a stochastic decision-making process that formally models the transition probabilities from a state s to a subsequent state s' provided that action a is performed, expressed mathematically as $P(s'|s, a)$ [7, 59].

The objective is to learn a policy $\pi(a|s)$ that maximizes the expected long-term cumulative reward, defined as $\mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t r_t\right]$. The discount factor $\gamma \in [0, 1]$ determines the importance of future rewards compared to immediate rewards. A value of γ close to 0 makes the agent focus more on immediate rewards, while a value close to 1 encourages the agent to consider long-term rewards more heavily [7, 60].

A MDP also defines this reward function that specifies the reward r an agent receives when taking action a in state s and transitioning to a subsequent state s'. This is commonly expressed as $R_a(s, s')$ [7, 59].

The **Markov property** says that all information from the past is contained in the current state, mathematically expressed as $P(s_{t+1}|s_t) = P(s_{t+1}|s_1, s_2, ...s_t)$. In other words, the transition probabilities depend only on the current state and not on the history of previous states [60].

In practice, the Markov property is often not strictly satisfied. For example, a single image captured by a camera does not provide sufficient information to infer the dynamics of the environment. From one image alone, it is not possible to determine whether a car is moving forward at 100 km/h or 10 km/h, reversing, or remaining stationary. However, such information is crucial for making correct decisions in applications like autonomous driving [61]. Therefore, in practice, only sufficiently informative state representations are often used because the ideal Markov property is not strictly fulfilled. This is often achieved by incorporating a history of observations, such as a sequence of the last five camera frames, to better capture the underlying dynamics of the environment [59].

2.8.2 Training Process

At the beginning of the training process, the policy is typically initialized randomly, meaning that the robot initially performs random movements. Through interaction with the environment, the agent explores different actions and attempts to identify those that yield high rewards. Actions associated with higher rewards are subsequently assigned a higher probability by the policy, making the robot more likely to select them in similar situations in the future.

A successful learning process requires a suitable balance between exploration and exploitation. Exploration refers to the evaluation of previously untried or infrequently selected actions in a given state, whereas exploitation denotes the repeated selection of actions that have already been identified as beneficial. During the early stages of training, greater emphasis is placed on exploration, as the agent must first acquire knowledge about the environment and determine which actions are most effective for accomplishing the task. As training progresses and the agent gains a better understanding of the environment and its reward structure, exploitation becomes increasingly important. This allows the agent to more frequently select actions that have proven successful, thereby improving overall task performance.

Numerous methods and algorithms have been proposed to effectively balance exploration and exploitation. Common examples include the ϵ-greedy strategy, Upper Confidence Bound (UCB), and Thompson Sampling. However, a detailed discussion of these methods is beyond the scope of this thesis.

Due to the exploration phase, the success of the chosen RL approach only becomes visible at a late stage, unlike in classical deep learning, see Figure 2.23.

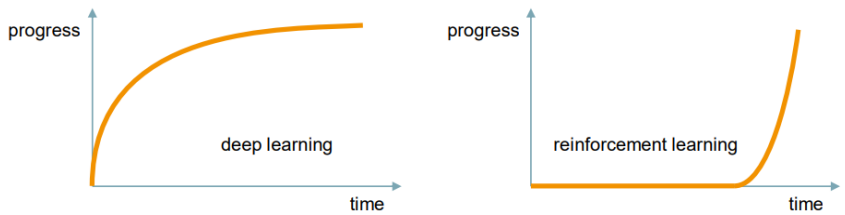


Figure 2.23: Training Progress in Deep Learning and Reinforcement Learning

2.8.3 Knowledge Modality from Video

An RL Knowledge Modality (KM) is some function learned from data that represents specific types of RL-related knowledge. KMs can be learned through trial-and-error principles or from various data sources, like video datasets, and can capture different aspects of the RL problem [62].

KM from Video:

- *Policy:* A Policy $\pi(a_t|s_t)$ is a mapping from states to actions and defines the behavior of the agent that is in a particular state [7]. $\pi(a_t|s_t)$ is the probability of taking action a_t given that the agent is in state s_t. t denotes the time step. In other words, the policy specifies how likely the agent is to select a particular action given its current observation of the environment. It can be learned from video data by observing the actions taken in different states and learning to predict those actions based on the visual input [62].
- *Value functions:* Value functions estimate the expected future return (cumulative reward) for a given state or state-action pair [7]. They can be learned from video data by observing the outcomes of actions and learning to predict the expected future rewards based on the visual input [62].
  - *State-value function (V-function):* A V-function $V_\pi(s_t)$ is a mapping from states to the expected future return when acting under a particular policy π [7].

    Intuition: "How good is it to be a certain state, if I follow my policy π?"

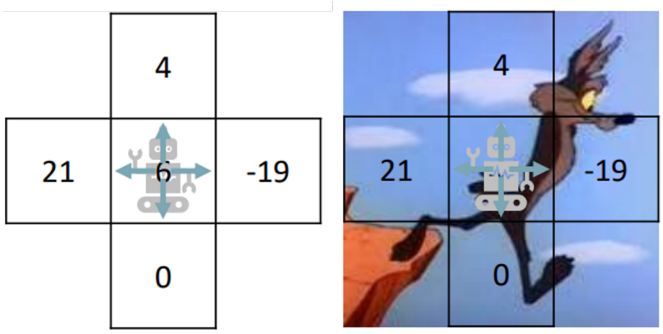


Figure 2.24: State-Value Function

  - *State-action value function (Q-function):* A Q-function $Q_\pi(s_t, a_t)$ is a mapping from state-action pairs to the expected future return when acting under a particular policy π [7].

    Intuition: "How good is this specific action here, if I continue behaving like my policy π?"

  - *V-function vs. Q-function:* The value function $V_\pi(s)$ estimates the expected return of being in a particular state s while following the policy π [7]. However, it does not provide information about the quality of individual actions available in that state s. In contrast, the State-action value function $Q_\pi(s, a)$ evaluates the expected return of taking a specific action a in state

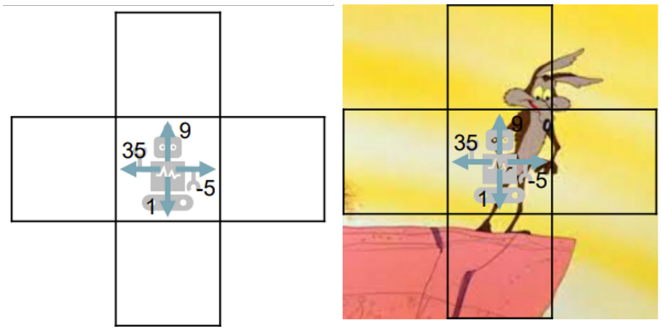


Figure 2.25: State-Action Value Function

s and subsequently following the policy π [7]. Therefore, the Q-function offers a more detailed assessment by considering both the state and the chosen action. Figures 2.24 and 2.25 illustrate the concepts of the state-value function and the action-value function, respectively.

- *Dynamics model:* A Dynamics model $p(s_{t+1}|s_t, a_t)$ predicts the next state s_{t+1} given the current state s_t and action a_t [7]. Video prediction models can be used as robot dynamics models [6]. By learning to predict future video frames based on current observations and actions, the model can capture the underlying dynamics of the environment. This allows the robot to anticipate the consequences of its actions and make informed decisions based on predicted future states.
- *Reward model:* A reward function $r(s_t, a_t)$ defines the reward received by the agent for taking action a_t in state s_t [7]. It can be learned from video data by observing the outcomes of actions and inferring the underlying reward structure that motivates those actions. For example, if a video shows a robot successfully picking up an object, the reward function can learn to assign a positive reward to that action in similar states.

2.9 Imitation Learning

IL or Learning from Demonstration is a general term for methods in which the agent derives or learns its behavior from expert demonstrations [63].

IL makes it possible to use "in the wild" videos—such as those found in great variety on the internet, for example on platforms like YouTube — to train AI models [5].

The demonstration videos can show a robot, a human, or even a loose sequence of actions, both in the real world or in a simulation environment, performing a specific task. However, videos without a clearly defined objective or with unknown activity can also be used [5].

This allows models to be trained to capture the visual and semantic relationships, as well as the physical dynamics of the real world. This, in turn, is a key prerequisite for the development of a generalist policy. IL can be divided into Behavior Cloning and Learning from Videos/Observation.

2.9.1 Behavior Cloning

Behavior Cloning (BC) is a fundamental approach within IL, where a policy is learned directly from **annotated expert demonstrations** [6]. The training data consists of state-action pairs (s_t, a_t), and the problem is formulated as a standard supervised learning task.

Conceptually, BC can be understood as a form of direct imitation ("direct copying") of expert behavior, without explicitly modeling the underlying system dynamics or task objectives. Due to its simplicity and scalability, BC is one of the most widely used methods in robot learning from datasets.

Purpose: Policy Learning

BC learns a policy by supervised learning on expert demonstrations:

$$\pi_\theta(a_t \mid s_t) \tag{2.18}$$

where:

- s_t: State / Observation (images, proprioception, etc.) at time t
- a_t: Expert action at time t

The Policy $\pi_\theta(a \mid s)$ is optimized by minimizing the prediction error between the actions generated by the model and the corresponding expert actions.

$$\min_\theta \mathbb{E}_{(s,a)\sim\mathcal{D}} \left[\mathcal{L}(\pi_\theta(s), a)\right] \tag{2.19}$$

where:

- $\pi_\theta(s)$: The action your robot's policy predicts given state s.
- a: The actual action the expert took in that same state.
- $\mathcal{L}$: Usually Mean Squared Error (MSE) for continuous control or Cross-Entropy for discrete actions.
- $\mathbb{E}_{(s,a)\sim\mathcal{D}}[\cdot]$: Take all the state-action pairs (s, a) inside the expert dataset $\mathcal{D}$, and calculate the average value of the expression inside the brackets.

2.9.2 Learning from Videos / Observation

LfV, also referred to as Learning from Observation (LfO), is an IL approach that relies solely on **video demonstrations without explicit action annotations** [5, 64]. In contrast to BC, where state-action pairs are directly available, LfV aims to infer the underlying actions or high-level intent purely from perceptual data, i.e. mapping observations to actions $(s \to a)$. It often uses **Representation Learning**, to learn and discover the most important (unknown) underlying data characteristics by transforming and reducing input in a fully data-driven and automatic manner.

A central challenge in LfV is the **inference problem**: determining which actions correspond to the observed behavior in the video [5]. Since no action labels are provided, additional methods and techniques are required to recover or approximate the missing action information. See chapter **XXX** for more information.

Due to this property, LfV can be seen as a **weakly supervised or self-supervised Imitation Learning** approach. Its main advantage lies in scalability, as it enables leveraging large amounts of unannotated video data, including internet videos or human demonstrations, without the need for costly data annotation [5]. Furthermore, additional actions can be derived from annotated video data too [5].

In robotics, LfV is commonly used as a **pretraining step**. The learned representations capture visual, semantic, and dynamical aspects of the environment, which can later be refined using methods such as BC with annotated data or in RL [5]. This combination allows models to first learn the real worlds dynamics from large-scale video data and subsequently specialize on task-specific robot demonstrations.

Purpose: Inferring Actions

LfV learns to infer actions from observations in a weakly supervised or self-supervised manner.

$$a_t = f(s_t, s_{t+1}) \tag{2.20}$$

where:

- a_t: The "latent" or predicted action.
- s_t: The current state/video frame.
- s_{t+1}: The next state/video frame.
- f: The inverse dynamics function (often a neural network) that maps state transitions to actions.

Acronyms

BC Behavior Cloning

CNN Convolutional Neural Network

IL Imitation Learning

KM Knowledge Modality

LfO Learning from Observation

LfV Learning from Videos

MDP Markov decision process

ML Machine Learning

MLP Multi-Layer Perceptron

MSE Mean Squared Error

RL Reinforcement Learning

RNN Recurrent Neural Network

VLA Vision-Language-Action

Bibliography

[1] I. I. F. of Robotics. "World Robotics 2025 report – INDUSTRIAL ROBOTS – released by IFR", IFR International Federation of Robotics. (), [Online]. Available: https://ifr.org/ifr-press-releases/news/global-robot-demand-in-factories-doubles-over-10-years (visited on 06/17/2026).

[2] M. I. Robots. "What Are Industrial Robots? Learn the What's and Why's Now". (11/04/2024), [Online]. Available: https://mobile-industrial-robots.com/blog/what-are-industrial-robots-learn-the-what-s-and-why-s-now (visited on 06/17/2026).

[3] B. Siciliano and O. Khatib, Eds., *Springer Handbook of Robotics* (Springer Handbooks). Cham: Springer International Publishing, 2016, ISBN: 978-3-319-32550-7 978-3-319-32552-1. DOI: 10.1007/978-3-319-32552-1. [Online]. Available: https://link.springer.com/10.1007/978-3-319-32552-1 (visited on 06/17/2026).

[4] "Large Video Planner: A New Foundation Model for General-Purpose Robots," Kempner Institute. (), [Online]. Available: https://kempnerinstitute.harvard.edu/research/deeper-learning/large-video-planner-a-new-foundation-model-for-general-purpose-robots/ (visited on 06/17/2026).

[5] R. McCarthy *et al.*, "Towards generalist robot learning from internet video: A survey," *Journal of Artificial Intelligence Research*, vol. 83, 07/2025, ISSN: 1076-9757. DOI: 10.1613/jair.1.17400. [Online]. Available: http://dx.doi.org/10.1613/jair.1.17400.

[6] C. Eze and C. Crick, "Learning by watching: A review of video-based learning approaches for robot manipulation," *IEEE access : practical innovations, open solutions*, vol. 13, pp. 184 071–184 109, 2024. [Online]. Available: https://api.semanticscholar.org/CorpusID:267627541.

[7] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA: MIT Press, 2018.

[8] J.-W. Lin, "Artificial neural network related to biological neuron network: A review," 2017. [Online]. Available: https://api.semanticscholar.org/CorpusID:60402092.

[9] W. S. McCulloch and W. Pitts, "A logical calculus of the ideas immanent in nervous activity," *Bulletin of Mathematical Biology*, vol. 52, pp. 99–115, 1990. [Online]. Available: https://api.semanticscholar.org/CorpusID:15619658.

[10] F. Rosenblatt, "The perceptron: A probabilistic model for information storage and organization in the brain.," *Psychological review*, vol. 65 6, pp. 386–408, 1958. [Online]. Available: https://api.semanticscholar.org/CorpusID:12781225.

[11] M. Minsky *et al.*, "An introduction to computational geometry," 1969. [Online]. Available: https://api.semanticscholar.org/CorpusID:118679952.

[12] D. L. Yse. "Your guide to Perceptrons", Medium. (03/22/2024), [Online]. Available: https://lopezyse.medium.com/your-guide-to-perceptrons-f6763663e19 (visited on 06/15/2026).

[13] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.

[14] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning representations by back-propagating errors," *Nature*, vol. 323, pp. 533–536, 1986. [Online]. Available: https://api.semanticscholar.org/CorpusID:205001834.

[15] "Neural networks – TikZ.net." (04/12/2024), [Online]. Available: https://tikz.net/neural_networks/ (visited on 06/15/2026).

[16] K. Hornik, "Approximation capabilities of multilayer feedforward networks," *Neural Networks*, vol. 4, pp. 251–257, 1991. [Online]. Available: https://api.semanticscholar.org/CorpusID:7343126.

[17] D. Hendrycks and K. Gimpel, "Gaussian error linear units (gelus)," *arXiv: Learning*, 2016. [Online]. Available: https://api.semanticscholar.org/CorpusID:125617073.

[18] V. Nair and G. E. Hinton, "Rectified linear units improve restricted boltzmann machines," in *International Conference on Machine Learning*, 2010. [Online]. Available: https://api.semanticscholar.org/CorpusID:15539264.

[19] A. L. Maas, "Rectifier nonlinearities improve neural network acoustic models," 2013. [Online]. Available: https://api.semanticscholar.org/CorpusID:16489696.

[20] T. D. Ryck, S. Lanthaler, and S. Mishra, "On the approximation of functions by tanh neural networks," *Neural networks : the official journal of the International Neural Network Society*, vol. 143, pp. 732–750, 2021. [Online]. Available: https://api.semanticscholar.org/CorpusID:233296667.

[21] P. Ramachandran, B. Zoph, and Q. V. Le, "Swish: A self-gated activation function," *arXiv: Neural and Evolutionary Computing*, 2017. [Online]. Available: https://api.semanticscholar.org/CorpusID:196158220.

[22] L. Bottou, F. E. Curtis, and J. Nocedal, "Optimization methods for large-scale machine learning," *ArXiv*, vol. abs/1606.04838, 2016. [Online]. Available: https://api.semanticscholar.org/CorpusID:3119488.

[23] M. Lanham, *Learn ARCore - Fundamentals of Google ARCore*, ISBN: 978-1-78883-040-9. [Online]. Available: https://www.oreilly.com/library/view/learn-arcore/9781788830409/e24a657a-a5c6-4ff2-b9ea-9418a7a5d24c.xhtml (visited on 06/15/2026).

[24] A. Ananthaswamy. "Researchers Build AI That Builds AI", Quanta Magazine. (01/25/2022), [Online]. Available: https://www.quantamagazine.org/researchers-build-ai-that-builds-ai-20220125/ (visited on 06/15/2026).

[25] L. Bottou, "Large-scale machine learning with stochastic gradient descent," in *International Conference on Computational Statistics*, 2010. [Online]. Available: https://api.semanticscholar.org/CorpusID:115963355.

[26] H. E. Robbins, "A stochastic approximation method," *Annals of Mathematical Statistics*, vol. 22, pp. 400–407, 1951. [Online]. Available: https://api.semanticscholar.org/CorpusID:16945044.

[27] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, pp. 2278–2324, 1998. [Online]. Available: https://api.semanticscholar.org/CorpusID:14542261.

[28] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," *Communications of the ACM*, vol. 60, pp. 84–90, 2012. [Online]. Available: https://api.semanticscholar.org/CorpusID:195908774.

[29] S. H. Khan and R. Iqbal, "A comprehensive survey on architectural advances in deep cnns: Challenges, applications, and emerging research directions," *ArXiv*, vol. abs/2503.16546, 2025. [Online]. Available: https://api.semanticscholar.org/CorpusID:277244867.

[30] A. Younesi, M. Ansari, M. Fazli, A. Ejlali, M. Shafique, and J. Henkel, "A comprehensive survey of convolutions in deep learning: Applications, challenges, and future trends," *IEEE access : practical innovations, open solutions*, vol. 12, pp. 41 180–41 218, 2024. [Online]. Available: https://api.semanticscholar.org/CorpusID:267897956.

[31] L. Long and X. Zeng, "Convolutional neural networks," in *Beginning Deep Learning with TensorFlow: Work with Keras, MNIST Data Sets, and Advanced Neural Networks*, Berkeley, CA: Apress, 2022, pp. 361–460, ISBN: 978-1-4842-7915-1. DOI: 10.1007/978-1-4842-7915-1_10. [Online]. Available: https://doi.org/10.1007/978-1-4842-7915-1_10.

[32] "Understanding Filters in Convolutional Neural Networks (CNNs) | DigitalOcean". (), [Online]. Available: https://www.digitalocean.com/community/tutorials/filters-in-convolutional-neural-networks (visited on 06/12/2026).

[33] V. Dumoulin and F. Visin, "A guide to convolution arithmetic for deep learning," *ArXiv*, vol. abs/1603.07285, 2016. [Online]. Available: https://api.semanticscholar.org/CorpusID:6662846.

[34] "What are Convolutional Neural Networks? | IBM". (10/06/2021), [Online]. Available: https://www.ibm.com/think/topics/convolutional-neural-networks (visited on 06/12/2026).

[35] "What is a Convolutional Neural Network?", NVIDIA Data Science Glossary. (), [Online]. Available: https://www.nvidia.com/en-eu/glossary/convolutional-neural-network/ (visited on 06/12/2026).

[36] B. Hanin and M. Sellke, "Approximating continuous functions by ReLU nets of minimal width," *ArXiv*, vol. abs/1710.11278, 2017. [Online]. Available: https://api.semanticscholar.org/CorpusID:1681235.

[37] X. Jiang, M. Lu, and S.-H. Wang, "An eight-layer convolutional neural network with stochastic pooling, batch normalization and dropout for fingerspelling recognition of Chinese sign language," *Multimedia Tools and Applications*, vol. 79, no. 21, pp. 15 697–15 715, 06/01/2020, ISSN: 1573-7721. DOI: 10.1007/s11042-019-08345-y. [Online]. Available: https://doi.org/10.1007/s11042-019-08345-y.

[38] Swapna. "Convolutional Neural Network | Deep Learning," Developers Breach. (08/21/2020), [Online]. Available: https://developersbreach.com/convolution-neural-network-deep-learning/ (visited on 06/12/2026).

[39] A. Vaswani *et al.*, "Attention is all you need," in *Neural Information Processing Systems*, 2017. [Online]. Available: https://api.semanticscholar.org/CorpusID:13756489.

[40] "Attention in transformers, step-by-step | Deep ... | 3Blue1Brown". (), [Online]. Available: https://www.3blue1brown.com/lessons/attention/ (visited on 06/12/2026).

[41] "Transformer Explainer: LLM Transformer Model Visually Explained". (), [Online]. Available: https://poloclub.github.io/transformer-explainer/ (visited on 06/12/2026).

[42] S. Raschka, *Build a Large Language Model (from Scratch)*. Shelter Island, NY: Manning Publications, 2024, ISBN: 978-1-63343-716-6. [Online]. Available: https://www.amazon.com/Build-Large-Language-Model-Scratch/dp/1633437167.

[43] "Google Colab". (), Adresse: https://colab.research.google.com/github/tensorflow/tensor2tensor/blob/master/tensor2tensor/notebooks/hello_t2t.ipynb#scrollTo=OJKU36QAfqOC (besucht am 12. 06. 2026).

[44] A. Dosovitskiy *et al.*, "An image is worth 16x16 words: Transformers for image recognition at scale," *ArXiv*, vol. abs/2010.11929, 2020. [Online]. Available: https://api.semanticscholar.org/CorpusID:225039882.

[45] M. ud Din, W. Akram, L. S. Saoud, J. Rosell, and I. Hussain, "Vision language action models in robotic manipulation: A systematic review," *ArXiv*, vol. abs/2507.10672, 2025. [Online]. Available: https://api.semanticscholar.org/CorpusID:280252663.

[46] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," *2009 IEEE Conference on Computer Vision and Pattern Recognition*, pp. 248–255, 2009. [Online]. Available: https://api.semanticscholar.org/CorpusID:57246310.

[47] "Foundation Models. "Foundation models: Where data meets... | by Sonam Tripathi | Medium." (), [Online]. Available: https://tripathisonam.medium.com/fine-tuning-the-future-harnessing-the-potential-of-foundation-models-20263b97c895 (visited on 06/15/2026).

[48] R. Bommasani *et al.*, "On the opportunities and risks of foundation models," *ArXiv*, vol. abs/2108.07258, 2021. [Online]. Available: https://api.semanticscholar.org/CorpusID:237091588.

[49] T. B. Brown *et al.*, "Language models are few-shot learners," *ArXiv*, vol. abs/2005.14165, 2020. [Online]. Available: https://api.semanticscholar.org/CorpusID:218971783.

[50] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, "High-resolution image synthesis with latent diffusion models," *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 10 674–10 685, 2021. [Online]. Available: https://api.semanticscholar.org/CorpusID:245335280.

[51] A. Radford *et al.*, "Learning transferable visual models from natural language supervision," *ArXiv*, vol. abs/2103.00020, 2021. [Online]. Available: https://api.semanticscholar.org/CorpusID:231591445.

[52] O. J. Achiam *et al.*, "GPT-4 technical report," 2023. [Online]. Available: https://api.semanticscholar.org/CorpusID:257532815.

[53] M. Reid *et al.*, "Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context," *ArXiv*, vol. abs/2403.05530, 2024. [Online]. Available: https://api.semanticscholar.org/CorpusID:268297180.

[54] G. Team *et al.*, "Gemini: A family of highly capable multimodal models," 2023. arXiv: 2312.11805.

[55] T. Baltruaitis, C. Ahuja, and L.-p. Morency, "Multimodal machine learning: A survey and taxonomy," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 41, pp. 423–443, 2017. [Online]. Available: https://api.semanticscholar.org/CorpusID:10137425.

[56] P. P. Liang, A. Zadeh, and L.-p. Morency, "Foundations & trends in multimodal machine learning: Principles, challenges, and open questions," *ACM Computing Surveys*, vol. 56, pp. 1–42, 2022. [Online]. Available: https://api.semanticscholar.org/CorpusID:257038528.

[57] J. Ngiam, A. Khosla, M. Kim, J. Nam, H. Lee, and A. Ng, "Multimodal deep learning," in *International Conference on Machine Learning*, 2011. [Online]. Available: https://api.semanticscholar.org/CorpusID:352650.

[58] Walkerastro. "Action Space, State Space, Observation Space demystified", Medium. (08/26/2024), [Online]. Available: https://medium.com/@walkerastro41/action-space-state-space-observation-space-demystified-6c9c00a355b4 (visited on 04/28/2026).

[59] T. Miller. "Markov Decision Processes — Mastering Reinforcement Learning." (), [Online]. Available: https://gibberblot.github.io/rl-notes/single-agent/MDPs.html (visited on 04/28/2026).

[60] R. Jagtap. "Understanding the Markov Decision Process (MDP)", Built In. (), [Online]. Available: https://builtin.com/machine-learning/markov-decision-process (visited on 04/28/2026).

[61] C. Staff. "What Is a Markov Decision Process?", Coursera. (15. 05. 2025), Adresse: https://www.coursera.org/articles/what-is-a-markov-decision-process (besucht am 28. 04. 2026).

[62] M. Wulfmeier, A. Byravan, S. Bechtle, K. Hausman, and N. M. O. Heess, "Foundations for transfer in reinforcement learning: A taxonomy of knowledge modalities," *ArXiv*, vol. abs/2312.01939, 2023. [Online]. Available: https://api.semanticscholar.org/CorpusID:265609417.

[63] I. Chrysomallis and G. Chalkiadakis, "Imitation learning in the deep learning era: A novel taxonomy and recent advances," *ArXiv*, vol. abs/2511.03565, 2025. [Online]. Available: https://api.semanticscholar.org/CorpusID:282758415.

[64] R. Burnwal, H. Mehta, N. P. Bhatt, and B. Ravindran, "Learning from observation: A survey of recent advances," *ArXiv*, vol. abs/2509.19379, 2025. [Online]. Available: https://api.semanticscholar.org/CorpusID:281505412.

List of Figures

List of Tables